

Department of Electrical Engineering and Computer Science
University of Siegen
Chair of Interconnected Automation Systems
Prof. Dr.-Ing. Oliver Wallscheid

Master Thesis

Hardware accelerated embedded on-chip training for reinforcement and machine learning algorithms

by

Gururaj Shetty

Student ID: 1769113

Supervisor: Prof. Dr.-Ing. Roman Obermaisser

Filing Date: September 14, 2026

Eigenständigkeitserklärung

Ich versichere, dass ich die schriftliche Ausarbeitung selbständig angefertigt und keine anderen als die angegebenen Hilfsmittel benutzt habe. Alle Stellen, die dem Wortlaut oder dem Sinn nach (inkl. Übersetzungen) anderen Werken entnommen sind, habe ich in jedem einzelnen Fall unter genauer Angabe der Quelle (einschließlich des World Wide Web sowie generativer KI und anderer elektronischer Datensammlungen) deutlich als Entlehnung kenntlich gemacht. Dies gilt auch für angefügte Zeichnungen, bildliche Darstellungen, Skizzen und dergleichen. Ich nehme zur Kenntnis, dass die nachgewiesene Unterlassung der Herkunftsangabe als versuchte Täuschung gewertet wird.

Ich bestätige, dass die elektronische Version inhaltlich mit der gedruckten Version übereinstimmt.

14.09.2025

Datum



Unterschrift

Declaration of Authorship

I certify that I have prepared this written work independently and have not used any aids other than those specified. I have clearly identified all passages that are taken from other works in terms of wording or meaning (including translations) as borrowed material in each individual case, stating the exact source (including the World Wide Web as well as generative AI and other electronic data collections). This also applies to attached drawings, pictorial representations, sketches and the like. I acknowledge that the proven omission of the indication of origin will be regarded as attempted deception.

I confirm that the content of the electronic version is the same as the printed version.

Abstract

Embedded systems are used for control tasks that require local and timely decisions. Reinforcement-learning training introduces an additional computational demand because the controller must repeatedly interact with a system, test actions, receive feedback, and update its future decisions. These repeated computations can make efficient onboard training challenging.

This thesis investigates hardware-accelerated embedded reinforcement-learning training using a benchmark control problem. The benchmark represents an unstable system that must be balanced by selecting suitable control actions. It is used to evaluate whether this training process can be carried out directly on an embedded platform and whether the resulting controller can achieve stable control behaviour.

The implementation follows a hardware/software co-design approach on the ZCU104 platform. The software part manages the experiment flow, environment simulation, feedback calculation, and evaluation. The hardware part accelerates the repeated computations required to update the controller during training, so that flexible system control remains in software while the computationally intensive operations are moved to dedicated hardware.

The evaluation compares the embedded hardware-accelerated implementation with a processor-only reference. The results show that onboard training is feasible on the embedded platform and that the resulting controller stabilizes the benchmark system while reducing the complete training runtime compared with the processor-only implementation.

Contents

1	Introduction	1
2	Theoretical Background	3
2.1	Reinforcement Learning and Deep Q-Networks	3
2.1.1	Reinforcement-Learning Concepts	3
2.1.2	Inverted-Pendulum Benchmark	4
2.1.3	Action-Value Learning and Neural-Network Approximation	6
2.1.4	Deep Q-Network Training	7
2.2	Embedded FPGA-Based Systems	10
2.2.1	Embedded FPGA Platform: Processing System and Programmable Logic	10
2.2.2	High-Level Synthesis	11
2.2.3	AXI-Lite Communication and Memory-Mapped Registers	14
2.2.4	PYNQ-Based FPGA Control	14
2.2.5	FPGA Hardware Resources	15
3	Literature Review	16
3.1	Reinforcement-Learning Control Applications	16
3.2	FPGA-Based Inference for Learning-Based Systems	17
3.3	FPGA Acceleration of Reinforcement-Learning Workloads	17
3.4	Comparison of Related Work	18
3.5	Research Gap	20
4	System Design and Implementation	21
4.1	Implemented System Overview	21
4.2	Processor-Side Software Design	22
4.3	Programmable-Logic Accelerator Design	23
4.3.1	Internal Storage and Weight Organization	24
4.3.2	DQN Training and Inference Computation	25
4.4	Board Integration and PYNQ-Based Communication	26
4.4.1	ZCU104 Platform	27
4.4.2	Vivado Block Design and AXI-Lite Interface	28
4.4.3	PYNQ-Based PS-PL Communication	30
4.5	Runtime Operation	30
4.5.1	Training Behaviour	30

4.5.2 Validation Behaviour	32
4.5.3 Inference Behaviour	32
4.5.4 Trained-Parameter Saving and Reloading	32
5 Evaluation and Results	34
5.1 Evaluation Setup	34
5.2 Learning Behaviour and Control Evaluation	35
5.3 Runtime Comparison and Speedup Analysis	42
5.4 FPGA Resource Usage and Network Depth	46
6 Conclusion and Future Scope	47
6.1 Conclusion	47
6.2 Future Scope	48
Appendix	49
A.1 Artificial Intelligence Use Declaration	49
Lists	50
List of Tables	50
List of Figures	50
Acronyms	51
Symbols	52
References	54

1 Introduction

Embedded control systems are computing systems that are integrated into larger technical products or processes. They observe the state of a physical system, process the available information, and generate a suitable control response. Examples include motion-control systems, power-electronic converters, sensor-processing units, and automated machines. In these applications, computation is closely connected to the behaviour of the physical process. Therefore, decisions often have to be made locally while operating under limited time, power, and hardware resources.

As embedded applications become more complex, their computational requirements also increase. A general-purpose processor provides flexibility and can execute a wide range of software tasks. However, processor-only execution can become a bottleneck when an application contains repeated numerical operations. Hardware acceleration addresses this by implementing selected computations in dedicated hardware, where parallelism and application-specific data paths can reduce their execution time.

Embedded field-programmable gate array (FPGA)-based platforms are suitable for this purpose because they combine a processor with reconfigurable programmable logic. The processor can handle flexible software tasks, while the programmable logic can be adapted to operations that occur repeatedly. This makes such platforms relevant for embedded applications that require both flexible control and efficient computation.

Machine learning (ML) is increasingly used in control systems to make their behaviour more adaptive. Instead of relying only on fixed rules, an ML model can improve its behaviour from data collected during operation or simulation. In an artificial neural network (ANN), the input values are processed through connected layers of neurons with weights and biases. During training, these weights and biases are adjusted so that the network output becomes more suitable for the given task. After training, the model is used during inference to produce outputs for new inputs.

In many embedded ML applications, training and inference are separated. The model is trained on an external computer, and only the trained model is later deployed on the embedded device. This separation is common because training is usually more computationally demanding than inference. For embedded control applications, the main challenge is therefore not only whether a trained model can be executed on the embedded device, but

also whether the training process can be accelerated efficiently enough to make onboard execution practical.

This challenge is especially relevant for reinforcement learning (RL). In RL, a controller improves its action selection by interaction with an environment. The environment models the system or simulation where the controller performs actions. After performing an action, the controller receives feedback in the form of a reward, which evaluates how suitable the action was for the control objective. Over many repeated interactions, the controller updates its policy, which defines how actions are selected for given system states.

For control applications, RL is attractive because it can learn a control policy from interaction with the system model or environment. However, this also introduces practical challenges. Training requires many repeated interactions and many repeated policy updates. Direct interaction with a real plant can also raise safety concerns during exploration, because unsuitable actions may occur before a stable policy has been learned. In addition, the repeated neural-network computations required during training can lead to long execution times on a processor-only implementation.

The problem addressed in this thesis is how the tasks in an embedded RL training system can be identified and divided between software and hardware. Flexible tasks such as experiment control, environment simulation, reward calculation, validation, logging, and communication are suitable for software execution on the processing system. Repeated computational tasks such as neural-network forward computation, target-value calculation, backpropagation, parameter update, replay-buffer handling, and target-network synchronization are suitable for acceleration in programmable logic. A suitable hardware/software split is therefore required to support onboard training while reducing the total training runtime.

The proposed system is evaluated using a representative control benchmark. The evaluation focuses on onboard RL training capability, control performance after training, complete training runtime compared with a central processing unit (CPU)-based reference implementation, and FPGA resource usage.

2 Theoretical Background

This chapter focuses on the concepts needed to understand the embedded DQN training system developed in this thesis. It first introduces the reinforcement-learning principles, action-value approximation, and training procedure underlying the implemented DQN. It then presents the embedded FPGA platform and the hardware-acceleration concepts relevant to its implementation.

2.1 Reinforcement Learning and Deep Q-Networks

2.1.1 Reinforcement-Learning Concepts

Machine learning (ML) enables a model to improve its behaviour using data or experience. In the control task considered in this thesis, the suitable action is not given as a labelled target output. The controller must instead learn from repeated interaction with the simulated environment: it selects an action, observes the resulting state and reward, and uses this feedback to improve later decisions. This learning process forms the foundation of RL [1, 2].

At each time step t , the agent receives the current state s_t and chooses an action a_t . Once this action is applied, the environment provides the resulting reward r_{t+1} and next state s_{t+1} . The reward evaluates the resulting behaviour and provides the signal used for learning. In the implementation, one interaction is stored as the transition (s, a, r, s', d) , where s' is the state reached after applying a in s , and d indicates whether the transition ends the episode. An episode consists of a sequence of such transitions and ends when a termination condition or a predefined time horizon is reached.

These elements can be expressed as a Markov decision process, defined by a state space $\mathcal{S}$, an action space $\mathcal{A}$, a transition mechanism $\mathcal{T}$, a reward function $\mathcal{R}$, and a discount factor γ [1]. The state contains the information used by the agent to select an action, while the transition mechanism determines the next state resulting from the current state and action. In this thesis, the DQN does not learn or store a separate transition model; the simulated pendulum environment directly generates s_{t+1} and r_{t+1} . The specific state, actions, reward, and transition assumptions used for this benchmark are defined in Section 2.1.2.

The rule used by the agent to select actions is called a policy. A policy defines how the agent selects an action from the available action set for a given state. During training, this work uses an ϵ -greedy strategy to balance exploration and exploitation. A random action is chosen with probability ϵ to encourage exploration of the action space, whereas the remaining probability $1 - \epsilon$ is used to choose the action with the largest estimated Q-value according to the current network. Exploration is disabled during validation and final evaluation and only the greedy action is performed.

The goal is to learn a policy that gives a high accumulated reward. The discounted return from time step t is defined as [1]

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}, \quad (2.1)$$

where T is the episode length and $\gamma \in [0, 1]$ is the discount factor. A value of γ close to zero emphasizes immediate rewards, whereas a value close to one gives greater weight to future rewards.

Figure 2.1 illustrates the interaction used to collect the transitions required for learning.

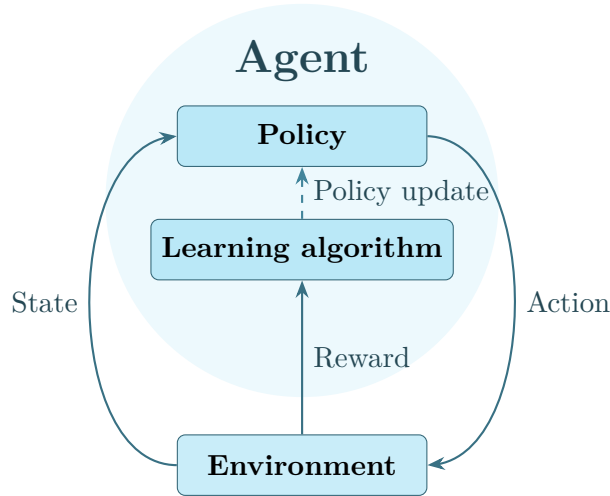


Fig. 2.1: Basic RL interaction between agent and environment.

2.1.2 Inverted-Pendulum Benchmark

The inverted pendulum is used as the benchmark for the hardware-accelerated training process. The aim is to apply a series of torque inputs to swing the pendulum from its initial state to upright position and maintain this equilibrium state. The system provides a compact nonlinear problem with continuous state variables [2, 3].

Figure 2.2 shows the basic model. The pendulum angle is denoted by θ , the angular velocity by $\dot{\theta}$, and the length by l . In the implemented environment, $\theta = 0$ modulo 2π represents the upright position and $\theta = \pi$ represents the downward position.

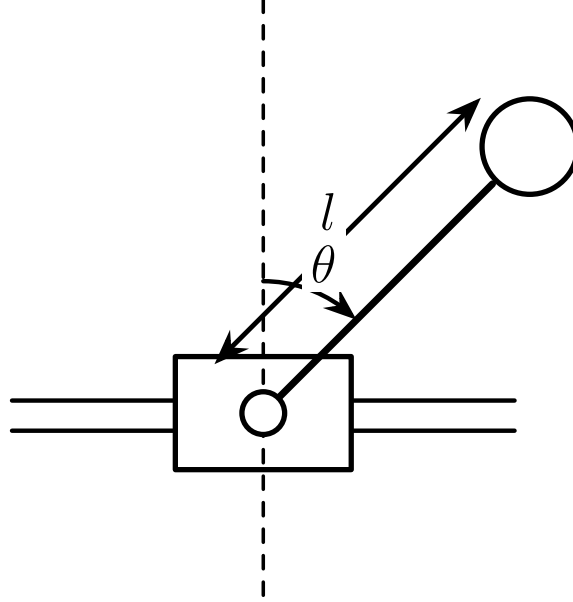


Fig. 2.2: Inverted pendulum model

The simulated pendulum has a mass of 1 kg, a length of 1 m, and a gravitational acceleration of 10 m/s^2 . These assumptions define the state transition calculated by the processor-side environment. At each time step, the processor-side environment first updates the angular velocity from the current angle and selected torque. The updated angular velocity is limited to the permitted range, after which it is used to update the angle. The continuous-time torque-driven pendulum model is based on the rigid-body dynamics described in [4]. The processor-side environment applies the following discrete-time update:

$$\begin{aligned}\dot{\theta}_{t+1} &= \text{clip}\left(\dot{\theta}_t + \left(\frac{3g}{2l} \sin \theta_t + \frac{3}{m_p l^2} u_t\right) \Delta t, -\dot{\theta}_{\max}, \dot{\theta}_{\max}\right), \\ \theta_{t+1} &= \theta_t + \dot{\theta}_{t+1} \Delta t.\end{aligned}\tag{2.2}$$

Here, m_p is the pendulum mass, l is its length, g is the gravitational acceleration, u_t is the applied torque, and Δt is the simulation time step. The angle is not explicitly limited; its periodicity is handled by representing it through $\cos \theta_t$ and $\sin \theta_t$ in the agent state.

The environment internally uses the angle and angular velocity. Before the state is passed to the agent, it is represented as

$$s_t = (\cos \theta_t, \sin \theta_t, \dot{\theta}_t). \quad (2.3)$$

Using $\cos \theta_t$ and $\sin \theta_t$ avoids a discontinuity between equivalent angular positions at the angle wraparound. The resulting state is continuous and has three components.

The action applied by the agent to the environment is a torque. Although the pendulum model accepts a continuous torque, the input is restricted to

$$\mathcal{A} = \{-2.0, -0.5, -0.1, 0.0, 0.1, 0.5, 2.0\}. \quad (2.4)$$

The agent selects one of seven action indices, and the selected index is mapped to the corresponding torque. This discretization makes the task compatible with DQN, which produces one value for each available action.

The reward favours an upright pendulum with low angular velocity and penalizes large torque values. It also rewards progress toward the upright position and adds cumulative bonuses within increasingly narrow upright regions. Let e_t denote the angle wrapped to $[-\pi, \pi]$, and let b_{t+1} denote these upright bonuses. The implemented reward is

$$r_{t+1} = \text{clip}\left(2 \cos e_{t+1} - 0.01 \dot{\theta}_{t+1}^2 - 0.001 u_t^2 + 0.5(|e_t| - |e_{t+1}|) + b_{t+1}, -10, 10\right). \quad (2.5)$$

The bonus b_{t+1} adds 2 when $|e_{t+1}| < 0.3$, a further 4 when $|e_{t+1}| < 0.15$ and $|\dot{\theta}_{t+1}| < 1.0$, and a further 6 when $|e_{t+1}| < 0.08$ and $|\dot{\theta}_{t+1}| < 0.5$. These conditions are cumulative, as in the processor-side reward function.

The clip operation limits the calculated reward to the interval $[-10, 10]$. A value above 10 is set to 10, a value below -10 is set to -10 , and a value within the interval remains unchanged.

The environment does not terminate an episode based on the pendulum state. Each episode instead uses a fixed horizon of 250 time steps. The continuous state and finite action set define the learning problem addressed by the DQN.

2.1.3 Action-Value Learning and Neural-Network Approximation

The action-value function $q^\pi(s, a)$ represents the expected discounted return obtained by selecting action a in state s and then following policy π [5]:

$$q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]. \quad (2.6)$$

Here, $\mathbb{E}_\pi$ denotes the expectation over the future states and actions generated while following policy π . For the implemented pendulum task, $q^\pi(s, a)$ estimates the expected

discounted future reward associated with each of the seven torque actions in the current state. q-learning uses the immediate reward and the largest next-state action value to improve this estimate [5]. This Q-learning update relation forms the basis of the target value used during DQN training.

As the pendulum state is continuous, storing a separate value for every state-action pair is impractical. A DQN therefore uses an ANN with trainable parameters $\mathbf{w}$ to approximate the action-value function as $q(s, a; \mathbf{w})$. The implemented network receives the state $(\cos \theta, \sin \theta, \dot{\theta})$ as three inputs, processes it through two hidden layers with 32 neurons each, and produces seven output Q-values corresponding to the seven discrete torque actions.

Each hidden neuron calculates a weighted sum of its inputs, adds a bias, and applies the ReLU function $f(x) = \max(0, x)$ [6]. The output layer is linear because it produces Q-values rather than class probabilities. During a forward pass, the network calculates all seven Q-values from the current state. The largest output is used for greedy action selection. The weights and biases are the trainable parameters that are modified during the DQN update.

Figure 2.3 shows how the state is mapped to the action values.



Fig. 2.3: Implemented Q-network mapping the three-component pendulum state to one Q-value for each of the seven discrete torque actions.

2.1.4 Deep Q-Network Training

During DQN training, the agent collects transitions while interacting with the environment. Rather than updating the network only from the most recent transition, the implementation stores each transition in a replay buffer. After the initial memory warmup, stored samples are randomly selected from this buffer and reused for training. This reduces the correlation between consecutive observations and allows earlier experiences to contribute to later parameter updates [1, 7].

The DQN maintains a main network and a target network during training. The main network is used to select actions and is updated from the replayed transitions. The target network is a delayed copy of the main network that provides the next-state value required for the training target. Its weights and biases are updated only periodically by copying them from the main network.

For a sampled transition $(s_i, a_i, r_i, s'_i, d_i)$, the implemented DQN target is [1, 7]

$$y_i = \begin{cases} r_i, & d_i = 1, \\ r_i + \gamma \max_{a'} q_{\text{target}}(s'_i, a'), & d_i = 0. \end{cases} \quad (2.7)$$

The accelerator implements both branches of this one-step Q-learning target. In the pendulum experiments, however, the environment uses a fixed episode horizon and supplies $d_i = 0$ for every environment step. The main network evaluates the corresponding current state, and only the output belonging to the sampled action is compared with the target. The TD error is [1]

$$\delta_i = y_i - q_{\text{main}}(s_i, a_i). \quad (2.8)$$

The corresponding squared TD loss can be written as [1]

$$L_i = \frac{1}{2} \delta_i^2. \quad (2.9)$$

Backpropagation uses the TD error to calculate an update direction g_t for the main-network weights and biases. The Adam optimizer maintains first- and second-moment estimates of this update direction and uses them to determine an adaptive update for each parameter. For a parameter w , the calculation is [8]

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \\ \widehat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \widehat{v}_t = \frac{v_t}{1 - \beta_2^t}, \\ w_t &= w_{t-1} + \alpha \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon_{\text{Adam}}}. \end{aligned} \quad (2.10)$$

Here, α is the learning rate, β_1 and β_2 are the moment-decay factors, and $\varepsilon_{\text{Adam}}$ prevents division by zero. The implementation forms g_t as an update direction from the TD error $\delta_t = y_t - q(s_t, a_t)$. Therefore, the parameter update uses a plus sign rather than the conventional minus sign used when g_t denotes the loss gradient. Adam updates the parameters of the main network, whereas the target-network parameters are updated at fixed intervals by copying the main-network parameters.

The accelerator executes an Adam parameter update for each sampled transition in the replay-memory group. The group size determines how many replay-based parameter updates are performed whenever the training condition is satisfied. Algorithm 2.1 summarizes the implemented DQN training procedure.

Algorithm 2.1 DQN training procedure used in this thesis

Input: differentiable action-value approximation $\hat{q}(s, \cdot; \mathbf{w})$, parameterized by $\mathbf{w}$
Parameters: exploration probability ϵ , discount factor γ , replay capacity, group size B , learning-start threshold, training interval, and target-update interval k_w
Initialization: initialize main-network parameters $\mathbf{w}$; set target-network parameters $\mathbf{w}^- \leftarrow \mathbf{w}$; initialize replay memory $\mathcal{D}$ and environment-step counter $n \leftarrow 0$

```

1: for  $j = 1, 2, \dots$  episodes do
2:   Initialize the environment and observe  $s_0$ 
3:   for  $k = 0, 1, \dots, T - 1$  do
4:     Select  $a_k$   $\epsilon$ -greedily with respect to  $\hat{q}(s_k, \cdot; \mathbf{w})$ 
5:     Apply  $a_k$  and observe  $r_{k+1}$ ,  $s_{k+1}$ , and  $d_{k+1}$ 
6:     Store  $(s_k, a_k, r_{k+1}, s_{k+1}, d_{k+1})$  in  $\mathcal{D}$ 
7:     if  $|\mathcal{D}|$  has reached the learning-start threshold and  $n$  is divisible by the training interval then
8:       for  $i = 1, \dots, B$  do
9:         Sample  $(s_i, a_i, r_i, s'_i, d_i)$  uniformly from  $\mathcal{D}$ 
10:        Calculate  $y_i$  using Equation 2.7
11:        Calculate  $\delta_i$  using Equation 2.8
12:        Backpropagate  $\delta_i$  through the main network
13:        Immediately update  $\mathbf{w}$  using Adam
14:      end for
15:    end if
16:     $n \leftarrow n + 1$ 
17:    if  $n$  is divisible by  $k_w$  then
18:      Copy  $\mathbf{w}^- \leftarrow \mathbf{w}$ 
19:    end if
20:  end for
21: end for

```

Figure 2.4 illustrates the overall DQN training workflow. It shows how transitions collected from the environment are stored in replay memory and used by the main and target networks during training.

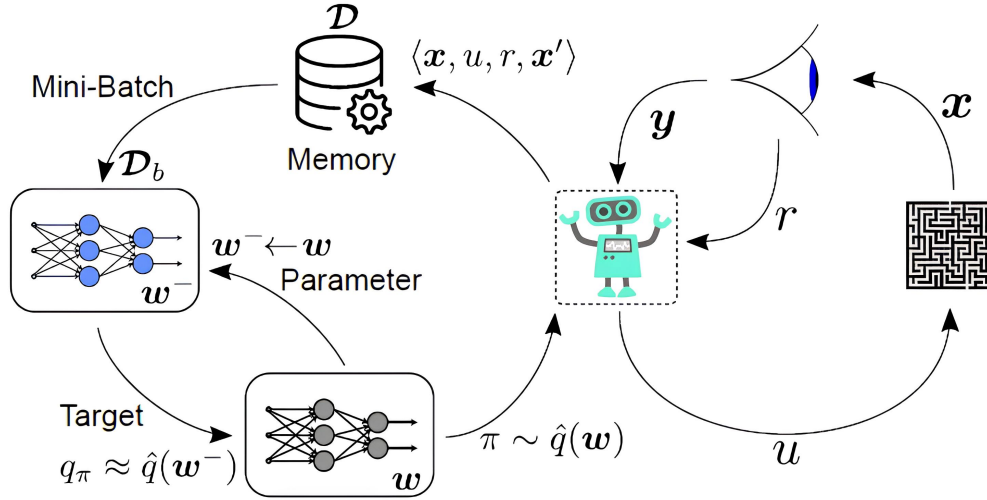


Fig. 2.4: DQN training workflow with replay buffer, target network, and parameter update. Adapted from [9].

2.2 Embedded FPGA-Based Systems

2.2.1 Embedded FPGA Platform: Processing System and Programmable Logic

An embedded FPGA platform can combine processor-based software execution and reconfigurable hardware in the same device. The processor-based part executes software and manages system-level tasks such as program control, memory access, and communication with peripherals. The reconfigurable hardware part contains the FPGA fabric, which can be configured to implement custom digital circuits and hardware accelerators.

AMD Xilinx Zynq devices combine a processor-based PS with reconfigurable PL on the same device. The PS provides processor cores and system-level components, whereas the PL provides configurable logic, arithmetic resources, on-chip memory blocks, and programmable routing [10].

This combination allows software and hardware-accelerated functions to work together in one embedded system. The PS is suitable for tasks that require flexibility, sequential control flow, operating-system support, and interaction with external interfaces. The PL is suitable for repeated computations that can benefit from application-specific hardware execution.

This architecture is important for embedded acceleration because not every part of an application has the same computational behaviour. Some tasks are easier to manage in software, while repeated numerical operations can be implemented as hardware accelerators in the programmable logic.

2.2.2 High-Level Synthesis

HLS converts an algorithm described in a high-level language, such as C or C++, into synthesizable hardware for FPGA implementation. In the AMD Xilinx design flow, Vitis HLS is used to generate synthesizable hardware from C/C++ descriptions [11].

Although the input to HLS is written in a software-like language, the result is not a software program running on a processor. The result is a hardware circuit. Therefore, the way the algorithm is written affects the generated hardware structure. Loops, arrays, functions, and memory accesses can influence resource usage, timing, and execution time [11].

HLS tools provide directives that guide how the hardware is generated. These directives can be used to control interfaces, loop execution, memory access, and storage mapping. Table 2.1 summarizes common HLS directives relevant to this thesis [11].

Tab. 2.1: Common HLS directives

Directive	Purpose
Pipeline	Allows different loop iterations to overlap
Unroll	Creates parallel hardware for loop iterations
Array partition	Splits arrays to increase parallel memory access
Interface	Defines how the hardware block communicates with other system parts
Bind storage	Maps arrays to specific FPGA memory resources

Pipelining is used to overlap the execution of operations or loop iterations. In sequential execution, one operation must finish before the next operation starts. In a pipelined design, different stages can process different data items at the same time. This improves throughput because a new operation can begin before the previous operation has completely finished [11].

Figure 2.8 illustrates the difference between sequential execution, task parallelism within one run, and task parallelism with pipelining. The figure shows that pipelining does not necessarily reduce the time of one individual stage, but it allows multiple stages or runs to overlap in time.

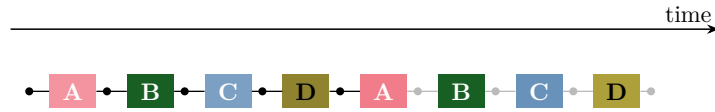


Fig. 2.5: Sequential Execution

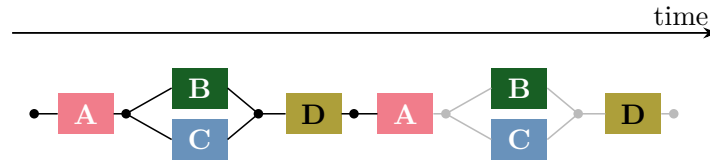


Fig. 2.6: Task Parallelism within a Run

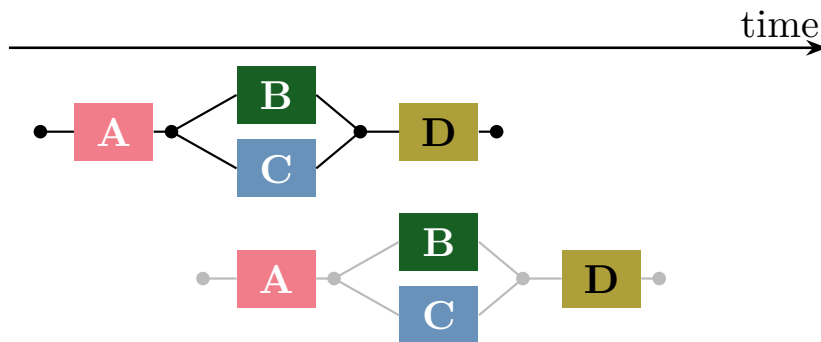
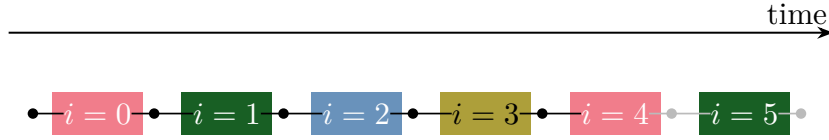
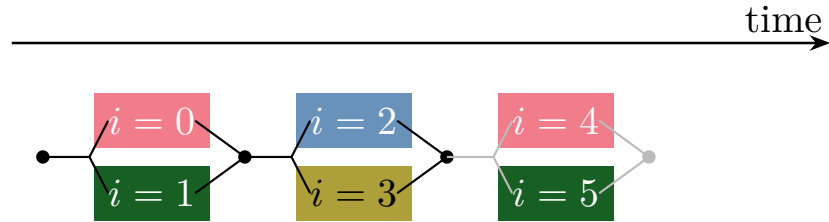


Fig. 2.7: Task Parallelism with Pipelining

Fig. 2.8: Conceptual illustration of sequential execution, task parallelism, and task parallelism with pipelining.

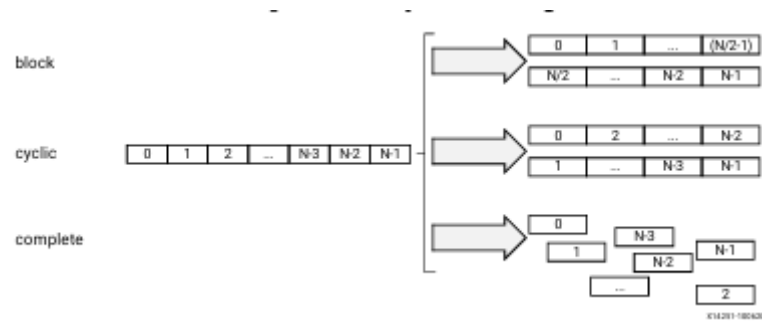
Loop unrolling is an HLS optimization in which the loop body is replicated so that several independent iterations can be executed at the same time [11]. Without unrolling, the iterations share the same hardware structure and are executed one after another. When the iterations are independent, unrolling can reduce the execution time by processing several iterations in parallel. This improvement requires additional FPGA resources because more arithmetic and logic units may be generated.

Figure 2.11 shows the difference between rolled loop execution and loop unrolling with an unroll factor of two. With an unroll factor of two, two loop iterations can be executed in parallel if no data dependency prevents parallel execution.

**Fig. 2.9:** Loop Execution without Unrolling**Fig. 2.10:** Loop Unrolling with Unroll Factor 2**Fig. 2.11:** Conceptual illustration of rolled loop execution and loop unrolling with an unroll factor of two.

Array partitioning changes how an array is mapped to hardware storage. Instead of storing the complete array as one memory structure, the array can be divided into smaller memories or registers. This allows multiple elements to be accessed in the same clock cycle. Array partitioning is important when pipelined or unrolled hardware requires several data values at the same time [11].

Common partitioning styles include block, cyclic, and complete partitioning. In block partitioning, consecutive sections of an array are grouped into separate blocks. In cyclic partitioning, array elements are distributed across different blocks in a repeating pattern. In complete partitioning, each element is implemented as a separate storage element.

**Fig. 2.12:** Conceptual illustration of block, cyclic, and complete array partitioning [11].

Task-level parallelism refers to the parallel execution of larger operations, functions, or processing stages. In HLS, this can be achieved when independent tasks are scheduled

so that their execution overlaps. This is different from loop unrolling. Loop unrolling creates parallel hardware inside a loop, while task-level parallelism allows larger functions or stages to operate at the same time [11].

These optimization concepts are useful because they show how HLS can generate hardware that differs from sequential software execution. However, each optimization introduces trade-offs. Pipelining, unrolling, array partitioning, and task-level parallelism can improve execution time, but they can also increase resource usage or make timing closure more difficult. Therefore, HLS design requires a balance between execution time, hardware resources, memory access, and timing requirements.

2.2.3 AXI-Lite Communication and Memory-Mapped Registers

In an embedded FPGA-based system, the processor and the hardware logic need a communication interface to exchange control information and results. A common method is memory-mapped input/output, or MMIO. In MMIO, hardware registers are assigned addresses within the processor address space. The processor can access these registers using ordinary read and write operations, similar to accessing memory locations.

Memory-mapped registers are commonly used to control a hardware block. For example, the processor can write input values, configuration values, or start commands to specific register addresses. The hardware block can then process the data and write status information or output values back to registers that the processor can read. This makes MMIO suitable for control-oriented communication between software and hardware.

AXI is a standard interface protocol used in many FPGA-based systems to connect processing systems, memories, and hardware components. AXI-Lite is a lightweight version of AXI that is mainly used for simple control and status communication. It is suitable for register-based access, such as mode selection, input values, start signals, and output values [11].

MMIO through AXI-Lite is useful because it provides a simple way for processor-side software to control hardware logic. AXI-Lite is mainly intended for small control transfers rather than large data movement.

2.2.4 PYNQ-Based FPGA Control

An API provides software functions that allow a program to access hardware or system services without directly managing all low-level details. In embedded FPGA systems, such an interface is useful because processor-side software must load hardware designs, configure hardware blocks, start execution, and read back results.

PYNQ is a Python-based framework for working with Zynq-based FPGA platforms. It provides a software interface for loading FPGA overlays and accessing hardware blocks

from Python. For register-based hardware blocks, PYNQ supports memory-mapped access, which allows software to write configuration values, start hardware execution, and read status or output values.

In this thesis, PYNQ is the API layer used by the processor-side Python program to communicate with the hardware implemented in the programmable logic.

2.2.5 FPGA Hardware Resources

An FPGA is built from configurable hardware resources that can be connected to form application-specific digital circuits. These resources define what can be mapped to the programmable logic, including logic operations, arithmetic operations, storage elements, and internal data paths. Unlike a fixed processor, an FPGA design uses these resources to create hardware structures that match the required computation [10].

The main FPGA resources used to describe implementation size are summarized in Table 2.2. These resources are important because different parts of a hardware design stress different parts of the FPGA fabric. Control logic mainly uses LUTs and FFs, arithmetic-heavy operations can require more DSPs, and designs with large internal data storage can require more BRAM or URAM.

Tab. 2.2: Main FPGA hardware resources

Resource	Function
LUT	Implements combinational logic functions
FF	Stores values between clock cycles
DSP	Performs arithmetic operations such as multiplication and accumulation
BRAM	Provides on-chip memory for data storage
URAM	Provides larger on-chip memory on supported FPGA devices
Interconnect	Connects logic, arithmetic units, memories, and interfaces

Resource utilization is used to check whether a design fits on the selected FPGA device. It also helps explain design trade-offs. For example, using more arithmetic units or storing more data on chip can improve execution behaviour, but it increases the amount of FPGA fabric used. Therefore, resource usage is considered together with execution time when evaluating an embedded FPGA implementation.

3 Literature Review

This chapter reviews research related to RL-based control, FPGA acceleration, and on-board learning. The aim is to identify the open research gap in embedded FPGA-based RL training for control applications.

3.1 Reinforcement-Learning Control Applications

RL has been applied to various control problems in order to study how policies can be learned through interaction with a dynamic system. Benchmark systems such as the inverted pendulum are commonly used because they provide an unstable nonlinear control task while still being simple enough for detailed analysis. Existing studies have used Q-learning, DQN-based methods, policy-gradient methods, and hybrid RL-control approaches for balancing, swing-up, and stabilization tasks [2, 3, 12–14]. These works show that RL can learn to control unstable benchmark systems.

Beyond benchmark problems, RL has also been investigated for practical control applications. In electrical-drive systems, deep RL and online RL have been used for current-control and real-world transfer tasks [15, 16]. These studies highlight additional practical requirements, such as safety, latency and deployment on embedded platforms. RL has also been combined with edge-computing support and FPGA-based neural-network inference for motor torque control [17].

Overall, the control-oriented literature shows that RL is a suitable method for both benchmark control tasks and practical control systems. However, most of these works focus on the achieved control behaviour, the learning strategy, or the transfer of a trained policy to real hardware. The complete training process is usually not the main focus from an embedded-hardware implementation perspective. Consequently, it remains unclear how the complete RL training loop, rather than only the trained policy, can be executed efficiently on an embedded FPGA platform.

3.2 FPGA-Based Inference for Learning-Based Systems

While RL control studies mainly focus on learning behaviour and control performance, another important area of work focuses on the hardware execution of already trained neural networks. In this context, FPGA-based accelerators are often used to reduce inference latency and improve energy efficiency. They achieve this by using customized datapaths, parallel execution, local memory, and reduced data movement [18–20].

In control applications, FPGA implementations have been used mainly for real-time inference of trained neural networks. For example, MLP-based motor-control implementations use the FPGA to execute the trained network during control operation. Similarly, FPGA-based neural-network inference in safe RL control pipelines uses hardware acceleration to select control actions from an already trained network, while the learning process is performed separately [17, 21]. In both cases, the hardware focus is the fast execution of an already trained model, not the complete learning update.

This distinction is vital since inference and training have different computational requirements. Inference mainly requires a forward pass through the trained network to calculate the output for a given input. Reinforcement-learning training additionally requires interaction samples to be stored, actions to be selected during exploration, target values to be calculated, prediction errors to be formed, and network parameters to be updated repeatedly. Therefore, inference-oriented FPGA acceleration does not by itself cover the complete hardware execution of an RL training workflow.

FPGA-based neural-network training has also been demonstrated outside RL. Supervised FPGA training for digit classification shows that forward computation, backpropagation, and testing can be implemented on FPGA hardware [22]. This supports the feasibility of FPGA-based neural-network training in general. However, supervised training learns from labelled input-output examples, whereas RL training learns from interaction and reward feedback.

3.3 FPGA Acceleration of Reinforcement-Learning Workloads

Several studies have moved from inference-only acceleration toward FPGA acceleration of RL workloads. Neural-network-based Q-learning acceleration demonstrates that both prediction and learning-related computation can be supported in hardware during interaction with an environment [23]. Other hardware/software accelerator architectures for Q-learning show that value-update operations and action-selection-related computations can benefit from custom datapaths, pipelining, and parallel execution [24].

These works establish that RL computations can be mapped to FPGA hardware. They also show that the hardware complexity depends on the type of RL method. In tabular

Q-learning, the action-value function is stored directly in a table and learning mainly consists of updating stored values. In neural-network-based RL, the value function or policy is represented by a trainable network. This introduces additional operations such as repeated forward computation, stored interaction data, backpropagation, and parameter updates. The remaining challenge is therefore not only to accelerate isolated RL operations, but to organize them into an embedded learning workflow.

Other FPGA-based RL accelerators address more complex training workloads and accelerator architectures. Some approaches use multiple parallel learning agents that interact with environments and contribute to shared model updates, thereby increasing the amount of training experience processed per unit time [25]. Distributed RL acceleration studies focus on how learning workloads can be mapped efficiently to FPGA resources across more than one processing unit or accelerator instance [26]. Hardware-efficient RL accelerators reduce resource cost through simplified network structures, optimized numerical representation, and hardware-aware design choices [27–29].

This group of work shows that FPGA platforms can support RL workloads beyond inference. It also shows that accelerator performance depends strongly on data flow, memory organization, numerical representation, and resource allocation. However, the reviewed approaches differ in their learning algorithms, accelerator structures, and application settings. Therefore, the design problem is not only whether individual RL operations can be accelerated, but how the full control-oriented learning flow is divided between software and programmable logic.

On-device FPGA learning studies further highlight the difficulty of implementing full neural-network-based RL training on resource-limited embedded hardware. Some approaches reduce this burden by replacing conventional DQN training with lighter online learning methods that are more suitable for FPGA resources [30]. This makes onboard learning more practical, but it also changes the training workflow and the type of update performed during learning.

Configurable DQN accelerators show that value-based RL training can benefit from FPGA hardware and can provide speedup over CPU execution [31]. This direction is close to embedded RL training because it addresses neural-network-based value updates rather than only inference. However, the focus is mainly on accelerating the learning workload itself, while the integration with an embedded control loop, validation procedure, runtime measurement, and resource evaluation remains less emphasized.

3.4 Comparison of Related Work

Table 3.1 compares the most relevant works with respect to their main idea, whether training is performed on the target hardware, and their relevance to the present work.

Tab. 3.1: Focused comparison of related work and research gap

Work	Main idea	Training focus / hardware role	Relevance to this thesis
Mnih et al. [1, 7]	Deep Q-learning with replay memory and target network.	Algorithmic basis for DQN training.	Provides the main learning method, but not an embedded FPGA implementation.
Su et al. [23]	FPGA-oriented neural-network RL control.	Hardware support for RL computation.	Shows FPGA relevance for RL control, but not a complete modern DQN training flow.
Cho et al. [25]	FPGA acceleration of deep RL computation.	Acceleration of deep-RL training workload.	Shows acceleration potential, but not onboard DQN training for embedded control.
Watanabe et al. [30]	On-device FPGA learning using a lightweight RL approach.	Lightweight on-board learning on an embedded FPGA.	Relevant to onboard learning, but uses a different training structure from DQN.
Hu et al. [27]	Hardware-efficient RL controller for efficient implementation.	Hardware-aware RL acceleration.	Shows resource-aware RL design, but targets a different algorithm and control setting.
Sutisna et al. [24]	FPGA acceleration for RL workloads.	Custom hardware support for value-update computation.	Demonstrates RL acceleration, but not full DQN training with replay and optimizer updates onboard.
Ge et al. [26]	FPGA-based acceleration for RL workloads.	Efficient mapping of RL workloads to FPGA resources.	Shows recent FPGA acceleration work, but embedded control-focused onboard training remains less developed.

The reviewed studies show that FPGA platforms have been used for accelerating RL and neural-network-based control computations. However, the focus of these works differs: some address efficient inference, some accelerate selected learning operations, and others study hardware-friendly controller structures.

3.5 Research Gap

The literature shows that the individual parts of the problem have been studied in separate research directions. RL has been applied to benchmark and practical control systems [2, 3, 12–16]. FPGA hardware has been used for neural-network inference in time-critical control and decision-making applications [17, 21, 32]. FPGA accelerators have also been proposed for RL workloads, including Q-learning, DQN-related acceleration, distributed learning, and hardware-efficient training-oriented methods [23–27, 30, 31].

The open issue is the integration of these directions into one embedded learning workflow. Control-oriented studies mainly evaluate learning behaviour and control performance. Inference-oriented FPGA studies accelerate trained models but leave training outside the main hardware workflow. RL accelerators show that learning-related computation can be mapped to FPGA hardware, but often focus on selected kernels, simplified learning methods, or accelerator-level performance.

A complete embedded RL training workflow requires the hardware/software system to handle action selection, experience storage, replay-based updates, neural-network training, validation, final behaviour evaluation, runtime measurement, and FPGA resource usage together. This combined view is less developed in the existing literature.

The present work addresses this gap by implementing and evaluating an embedded FPGA-based DQN training workflow for inverted pendulum control. The goal is not to introduce a new RL algorithm, but to study how an existing DQN-style training process can be mapped to an embedded FPGA platform and evaluated in terms of learning behaviour, execution time, and hardware resource usage.

4 System Design and Implementation

The implemented embedded DQN training system is built on the ZCU104 platform. It combines a processor-side Python program with a programmable-logic accelerator and uses an AXI-Lite register interface for communication between both parts. The chapter describes the processor-side control flow, the accelerator storage and computation structure, the board integration, and the runtime operation used during training, validation, inference, and parameter saving.

4.1 Implemented System Overview

The onboard training workflow is built as an interaction loop between the processor-side Python program and the programmable-logic accelerator. The processor executes the inverted-pendulum environment and applies the action selected by the accelerator. After each environment step, the processor calculates the reward, obtains the next state, checks the episode status, and sends this step result back to the accelerator.

The accelerator uses the received step result together with internally stored data to continue the DQN training process. During training, it stores the transition data and performs the hardware-side DQN update using its internal replay buffer, network parameters, and optimizer states. The accelerator also selects the next action and returns it to the processor for the next environment step.

Only step-level values are exchanged during the normal training loop. These include the current state or next state, reward, done flag, exploration value, control command, selected action, and status information. Larger data structures such as replay buffer, main-network weights, target-network weights, Adam optimizer states, and internal counters are kept inside the accelerator. This avoids transferring the complete training state between the processor and programmable logic at every time step.

The same hardware block is also used during validation and final evaluation. In these phases, the processor still executes the environment, but the accelerator only performs the forward computation of the trained main network and returns the action with the highest

Q-value. No replay-buffer storage, backpropagation, Adam update, or target-network synchronization is performed during validation or final evaluation.

Figure 4.1 illustrates the data flow between the processor-side program, the AXI-Lite register interface, and the programmable-logic accelerator in the implemented system.

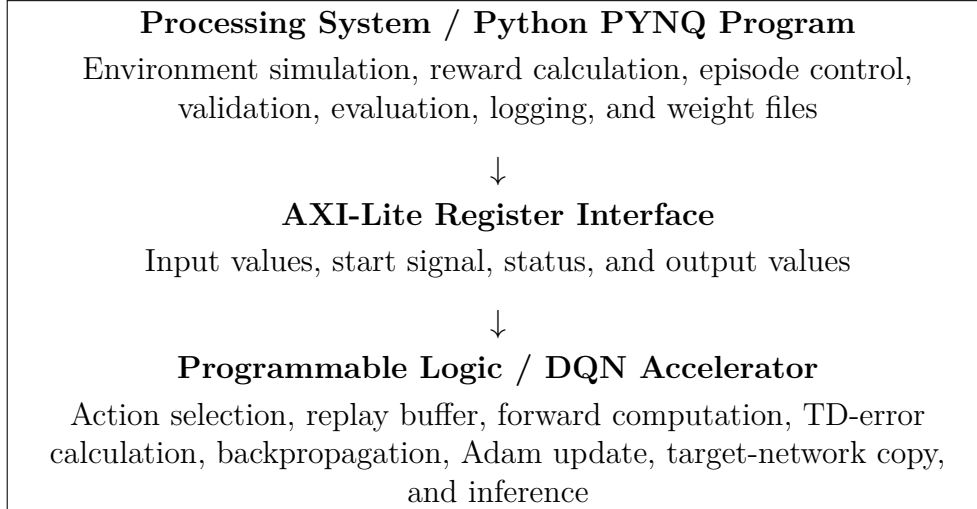


Fig. 4.1: Overview of the hardware/software structure used for embedded DQN training.

4.2 Processor-Side Software Design

A Python-based program runs on the processing system of the ZCU104 and provides the processor-side software functionality. Its purpose is to control the experiment around the hardware accelerator. It does not perform the neural-network training update during on-board training. Instead, it prepares the environment data, starts the required accelerator call, and stores the results produced during the experiment.

At the beginning of a run, the Python program loads the FPGA bitstream and creates a software handle to the accelerator register space through PYNQ. The program then prepares the training configuration used by the experiment, such as the episode counter, exploration, validation interval, output file paths, and runtime measurement variables. The accelerator is also prepared for a new run so that its internal memories, counters, and stored state-action values start from a defined condition.

The inverted-pendulum environment runs on the processor side. For every episode, the Python script initializes the environment and obtains the initial pendulum angle and angular velocity. These values are converted into the state representation used by the accelerator:

$$s = (\cos \theta, \sin \theta, \dot{\theta}).$$

The use of $\cos \theta$ and $\sin \theta$ avoids a discontinuity at the angle wraparound point, while $\dot{\theta}$ provides the angular-velocity information required for control.

During a training episode, the processor applies the action returned by the accelerator to the environment. After the environment step, the processor calculates the reward, obtains the next state, and checks the done flag. These values are written to the accelerator registers together with the current exploration value ϵ . The accelerator then performs the hardware-side part of the training step and returns the next action.

Training, validation, and final evaluation are executed as separate phases. During validation and final evaluation, the processor runs the environment and records the reward and stability values, while the accelerator only returns the greedy action from the trained main network. No replay-buffer entries are stored and no parameter updates are performed in these phases.

The processor also stores the experiment results, including episode rewards, validation rewards, final evaluation values, runtime measurements, and saved parameter files. During onboard training, the Python program does not modify the network weights. The working parameters remain inside the programmable-logic accelerator and are updated there.

4.3 Programmable-Logic Accelerator Design

The programmable-logic accelerator is implemented as a single Vitis HLS IP block. Its internal design combines register-controlled input and output logic, local training-state storage, main-network and target-network computation paths, replay-buffer access, gradient calculation, Adam-based parameter update, target-network synchronization, and action selection.

The processor selects the required accelerator operation through control registers. The same hardware block is used for training, validation, inference, and trained-parameter transfer. Table 4.1 gives an overview of these operating modes.

Tab. 4.1: Main accelerator operating modes

Mode	Function
Initialization	Prepares the internal accelerator state, counters, replay-buffer pointer, and stored state-action values for a new run.
First action selection	Receives the initial state of an episode, performs main-network forward computation, selects an action, and stores the initial state-action pair.
Training step	Receives reward, next state, done flag, and exploration value; forms a transition, stores it in the replay buffer, performs DQN updates when required, updates the target network when required, and returns the next action.
Validation / inference	Performs only main-network forward computation and greedy action selection. No replay-buffer write or parameter update is performed.
Trained-parameter save/reload	Reads trained main-network weights and biases from the accelerator after training and saves them on processor-side storage. For a later inference or evaluation run, the saved parameters are written back into the accelerator before execution starts.

4.3.1 Internal Storage and Weight Organization

The main-network weights and biases are stored in the programmable logic as layer-wise parameter arrays. Separate arrays hold the corresponding target-network parameters. During training, only the main-network arrays are updated; the target-network arrays are updated periodically by copying the current main-network parameters.

The replay buffer is implemented as internal transition storage. Each replay-buffer entry contains the state, selected action, reward, next state, and done flag. Since the replay buffer is the largest stored structure in the design, it dominates the on-chip memory usage, especially the URAM usage.

The HLS storage directives assign the large replay-state arrays to URAM. In this implementation, the arrays that store the replay states and replay next states are mapped to URAM, because they contain the state values for all stored transitions and therefore require the largest amount of on-chip storage. The smaller replay arrays, such as the selected-action and done-flag storage, are mapped to BRAM. The main-network weights and biases, target-network weights and biases, and Adam first- and second-moment arrays are also mapped to BRAM. These arrays are smaller than the replay-state storage, but they are accessed repeatedly during forward computation, backpropagation, and parameter updates.

The Adam optimizer states are stored inside the accelerator together with the trainable parameters. For each trainable main-network parameter, the corresponding first-moment and second-moment values are kept locally and updated together with the parameter value. The processor-side Python program does not store or update these weights or optimizer states during onboard training.

Array partitioning is applied to support parallel memory access during pipelined and partially unrolled computations. The small state arrays are completely partitioned, so the three state values can be accessed independently. The network-parameter arrays, target-network arrays, activation buffers, Q-value buffers, error buffers, and Adam moment arrays are cyclically partitioned using the selected parallel factor. This reduces memory-port conflicts when multiple neuron, action, or parameter values are processed in parallel.

The implemented accelerator uses a parallel factor of 2 for selected unrolled loops and cyclic array partitioning. This provides limited parallelism in the forward path, action-value comparison, backward computation, and parameter-update path while keeping the resource usage suitable for the ZCU104 device. The same factor is used for the corresponding partitioned parameter and optimizer-state arrays so that parallel arithmetic operations can access the required values without being limited by a single memory port.

The accelerator also stores the previous state-action pair, replay pointer, replay count, and training-step counters. The previous state-action pair is used to form a complete transition when the processor returns the reward, next state, and done flag after an environment step. The replay pointer and replay count manage the circular replay buffer, while the training-step counter is used to decide when training updates and target-network synchronization are performed.

4.3.2 DQN Training and Inference Computation

The accelerator computation is divided into hardware routines that operate on the internal replay buffer and parameter arrays. The main routines are the main-network forward computation, target-network forward computation, target-value calculation, error calculation, backward computation, Adam parameter update, target-network copy, and action selection. All routines are implemented within the same HLS IP core. The processor selects the required operation through the accelerator control interface.

The forward computation is implemented as layer-wise matrix-vector processing. For each layer, the accelerator reads the required input values, weights, and bias values from local arrays, calculates the weighted sums, and writes the layer output into local buffers. The hidden-layer outputs are passed through ReLU. These intermediate outputs are kept inside the accelerator because they are required again during the backward computation. The output layer produces the Q-values used either for action selection or for the training update.

The main network and target network use separate parameter arrays. Therefore, the accelerator can evaluate the current-state value with the main network and the next-state value with the target network without replacing one parameter set with the other. During a training update, the target-network forward computation is applied to the next state from a replay-buffer sample. The maximum output value is selected inside the accelerator and used in the target-value calculation. The main-network forward computation is applied to the corresponding stored state, and only the output belonging to the sampled action is forwarded to the error calculation.

The backward computation is implemented only for the main network. The calculated error is propagated from the selected output value towards the hidden layers. The accelerator reuses the stored activations from the forward computation to calculate the gradients for the weights and biases. For hidden layers, the ReLU result determines whether the corresponding neuron contributes to the gradient calculation. This keeps the backward path tied to the actually active part of the network for the sampled transition.

The parameter update is performed by the Adam update routine inside the accelerator. For each trainable main-network parameter, the routine reads the current weight or bias, the corresponding first-moment value, and the corresponding second-moment value. It then updates the moment values and writes the updated parameter back to the main-network array. The target-network arrays are not modified by this routine. They are updated only by the internal copy routine, which transfers the current main-network parameters into the target-network arrays when the target-update interval is reached.

For action selection, the accelerator executes the main-network forward computation for the current state and compares the output Q-values. During training, the selected greedy action is combined with the current ϵ value so that exploration can still be applied. During validation and final evaluation, exploration is disabled and the greedy action is returned directly. In these evaluation calls, the accelerator bypasses replay-buffer writing, target-value calculation, backward computation, Adam update, and target-network copying.

4.4 Board Integration and PYNQ-Based Communication

After the processor-side control program and the programmable-logic accelerator were defined, both parts were integrated on the ZCU104 platform. The integration connects the Python-based experiment flow running on the processing system with the custom DQN accelerator implemented in programmable logic. The Vivado block design provides the hardware connection, while PYNQ provides the software-side access to the mapped accelerator registers.

4.4.1 ZCU104 Platform

The implementation was carried out on the ZCU104 evaluation board shown in Figure 4.2. The board is based on a Zynq UltraScale+ MPSoC, consisting of ARM-based processing system and FPGA programmable logic. This device structure allows the processor-side control program and the programmable-logic accelerator to be integrated on the same embedded platform.

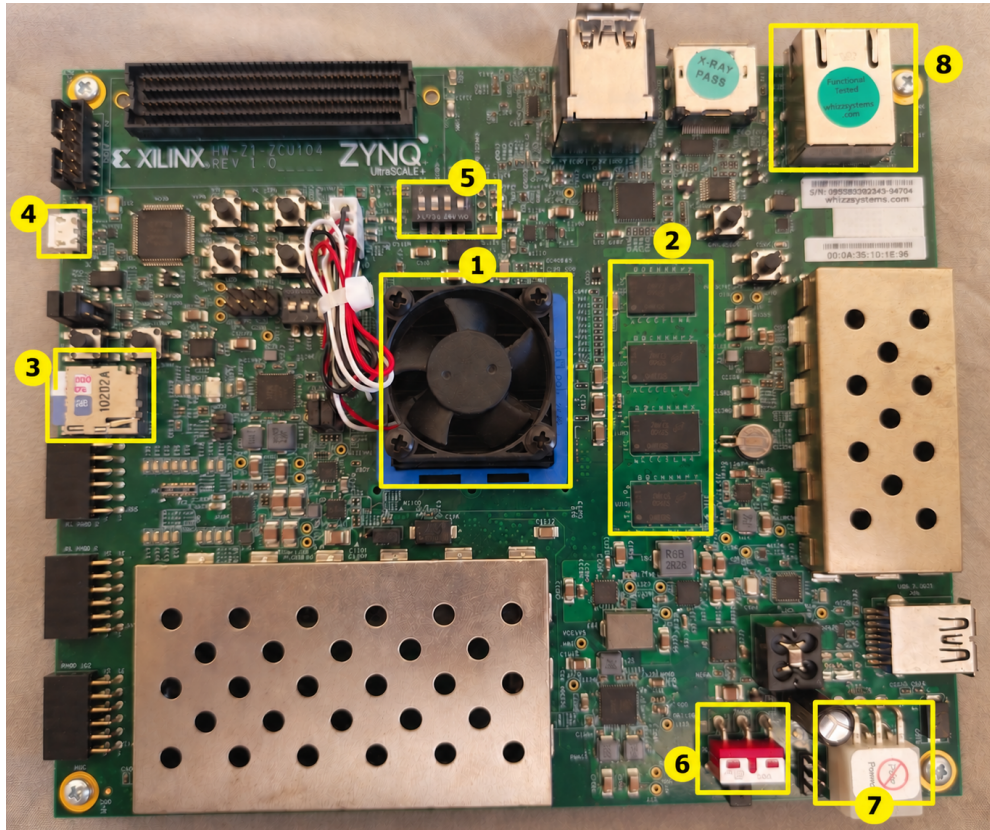


Fig. 4.2: ZCU104 evaluation board with implementation-relevant components marked: (1) Zynq UltraScale+ MPSoC, (2) PS-side DDR4 memory, (3) SD card connector, (4) USB/JTAG programming connection, (5) mode switch, (6) power switch, (7) power input, and (8) Ethernet/host connection.

The marked components in Figure 4.2 are the parts used directly or indirectly during implementation and testing. The Zynq UltraScale+ MPSoC provides the processing system and programmable logic used by the design. The PS-side DDR4 memory is used during Python execution. The SD card stores the boot image, Python scripts, bitstream, experiment logs, and saved parameter files. The USB/JTAG and Ethernet connections are used for programming, debugging, and board access. The power input, power switch, and mode switch are required for board operation and boot configuration.

The implementation-relevant platform details are summarized in Table 4.2. The table lists only the board features used in this thesis workflow, rather than the complete ZCU104 specification.

Tab. 4.2: Main platform details used in the implementation

Feature	Use in this implementation
Target board	ZCU104 evaluation board
Device family	Zynq UltraScale+ MPSoC
Processing system	Runs Linux, PYNQ, Python control program, and environment simulation
Programmable logic	Contains the custom DQN training accelerator
PS-side memory	DDR4 memory used by the processor-side software
Storage	SD card for boot files, Python scripts, bitstream, experiment logs, and saved parameter files
Development connection	USB/JTAG and Ethernet access during programming, debugging, and execution
Hardware interface	AXI-Lite register interface between processing system and accelerator
Design tools	Vitis HLS, Vivado, PYNQ, and Python

During the experiments, the board was booted into the PYNQ environment from the SD card. The generated bitstream was loaded before training, validation, or final evaluation was started. The accelerator then operated in programmable logic using the configured PL clock of 100 MHz.

4.4.2 Vivado Block Design and AXI-Lite Interface

After synthesis in Vitis HLS, the accelerator was exported as an RTL IP block and integrated into the Vivado block design shown in Figure 4.3. The block design connects the Zynq processing system to the custom DQN accelerator through an AXI-Lite interface.



Fig. 4.3: Vivado block design connecting the Zynq processing system with the custom DQN accelerator.

The main blocks in Figure 4.3 are the Zynq UltraScale+ processing system, the AXI SmartConnect, the Processor System Reset block, and the custom DQN HLS accelerator. The processing system acts as the AXI master. The accelerator is connected as an AXI-Lite slave peripheral. The AXI SmartConnect routes the register read and write transactions from the processing system to the accelerator.

The processor accesses the accelerator through the **M_AXI_HPM0_FPD** master interface of the Zynq processing system. The scalar arguments of the HLS top function are mapped to memory-mapped registers in the accelerator address space. These registers are used for operation selection, input state values, reward and done information, exploration value, start and completion control, returned action values, diagnostic values, and weight import/export data.

The AXI-Lite interface is used for control and step-level data transfer. During one accelerator call, the processor writes the required input values, starts the selected hardware operation, waits for completion, and reads the result. Larger training structures such as replay buffer, main-network parameters, target-network parameters, and Adam optimizer states are implemented inside the accelerator and are not moved through AXI-Lite during each training update.

The block design also provides the clock and reset connections required by the accelerator. The accelerator and AXI interconnect are driven by the programmable-logic clock configured from the processing system. The Processor System Reset block generates synchronized reset signals for the interconnect and the accelerator, so that the hardware starts from a defined state after bitstream loading or reset.

4.4.3 PYNQ-Based PS-PL Communication

The processor-side Python program controls the hardware design through PYNQ. After loading the bitstream, the program obtains access to the memory-mapped register space of the custom DQN IP. The same access method is used for training, validation, final evaluation, and parameter transfer.

A hardware call from Python follows a fixed register sequence. First, the processor writes the required input values to the mapped registers. Then it writes the operation selection and starts the accelerator. The Python program waits until the hardware completion flag is set and then reads the required output registers. During training, the returned value is the next selected action. During validation and final evaluation, the returned value is the greedy action. During parameter export, the returned value is the selected trained parameter.

During training, this register sequence is executed once per environment step. Only the step-level values are transferred through AXI-Lite, while the replay buffer, network parameters, and optimizer states remain inside the accelerator.

Direct memory-mapped register access is used in the Python code to reduce overhead during repeated accelerator calls. This is important because the training loop performs many step-level calls. Instead of transferring complete training data, the processor writes only the values needed for the current accelerator operation.

The same communication path is used during validation and final evaluation, but the selected accelerator operation is different. The processor writes the current state, starts the accelerator for action selection, and reads the greedy action returned by the main network. No replay-buffer write, target-value calculation, backpropagation, Adam update, or target-network copy is performed during these evaluation calls.

4.5 Runtime Operation

The runtime operation combines the processor-side environment loop with the accelerator calls described earlier. The following subsections focus on the order of calls and updates during training, validation, inference, and trained-parameter transfer.

4.5.1 Training Behaviour

At the start of a training run, the processor loads the bitstream, initializes the PYNQ overlay, prepares the accelerator register access, and prepares the accelerator for a new run.

For each episode, the processor initializes the environment and sends the initial state together with the current ϵ value to the accelerator. The accelerator selects the first action and stores the corresponding state-action pair internally.

During the step loop, the processor applies the selected action to the environment and sends the resulting reward, next state, and done flag back to the accelerator. The accelerator forms the transition, writes it into the replay buffer, performs the DQN update when the update conditions are satisfied, synchronizes the target network when required, and returns the next action.

Algorithm 4.1 summarizes this runtime sequence.

Algorithm 4.1 Training execution flow with processor and accelerator

```

1: Load bitstream and initialize PYNQ overlay
2: Prepare accelerator registers and reset experiment variables
3: Prepare accelerator internal state for a new training run
4: for each training episode do
5:   Reset environment on the processor side
6:   Read initial angle and angular velocity
7:   Convert environment state to accelerator state representation
8:   Write initial state and current  $\epsilon$  to accelerator
9:   Start accelerator for first action selection
10:  Read selected action from accelerator
11:  for each configured environment step do
12:    Apply selected action to the environment on the processor side
13:    Calculate reward  $r$ , next state  $s'$ , and done flag
14:    Write  $r$ ,  $s'$ , done flag, and current  $\epsilon$  to accelerator
15:    Start accelerator training-step operation
16:    Accelerator forms transition from stored previous state-action pair and received
    step result
17:    Accelerator writes transition into replay buffer
18:    if learning-start condition and training interval are satisfied then
19:      Accelerator samples replay-buffer entries
20:      Accelerator performs target-network forward computation
21:      Accelerator performs main-network forward computation
22:      Accelerator calculates target value and TD error
23:      Accelerator performs backward computation through main network
24:      Accelerator updates main-network parameters using Adam
25:    end if
26:    if target-update interval is reached then
27:      Accelerator copies main-network parameters to target-network arrays
28:    end if
29:    Accelerator selects next action using main-network output and  $\epsilon$ 
30:    Accelerator stores new state-action pair internally
31:    Processor reads next action from accelerator
32:  end for
33:  Store episode reward and training log values on the processor side
34: end for

```

The done flag is kept in the accelerator interface for compatibility with the general DQN update structure. In the implemented inverted-pendulum experiment, the environment does not terminate an episode based on a failure state. Instead, each episode uses the fixed horizon of 250 steps.

4.5.2 Validation Behaviour

Validation is executed during training to observe the behaviour of the current main network without modifying the training state. The processor resets the environment, sends the current state to the accelerator, applies the returned action, and records the validation reward.

In this phase, the accelerator only performs the main-network forward computation and Q-value comparison. It returns the greedy action to the processor. Replay-buffer writing, target-value calculation, backward computation, Adam update, and target-network synchronization are not executed during validation.

4.5.3 Inference Behaviour

Inference is performed after training using the trained main-network parameters. The processor runs the predefined evaluation cases and controls the inverted-pendulum environment in the same way as during validation. For each step, the processor converts the current environment state into the accelerator state representation and writes it to the accelerator.

The accelerator uses only the main-network forward computation during final inference. The output Q-values are compared inside the accelerator, and the action with the maximum Q-value is returned to the processor. The processor applies this action to the environment and records the reward, stable-step information, and final evaluation results.

During inference, the network parameters remain fixed. The accelerator does not write new transitions into replay buffer, does not calculate target values, does not perform backward computation, does not update Adam moment values, and does not synchronize the target network. Therefore, the final evaluation measures the behaviour of the trained main network without further learning.

4.5.4 Trained-Parameter Saving and Reloading

After onboard training, the trained main-network parameters are saved from the accelerator to processor-side storage. The processor reads the layer-wise weights and biases through the PYNQ-controlled AXI-Lite register interface and writes the received values as parameter files on the SD card. These files preserve the trained policy after the hardware training run has finished.

For a later evaluation or inference run, the processor reads the saved parameter files and writes the values back into the corresponding main-network parameter arrays inside the accelerator before execution starts. The loaded parameters are then used by the main-network forward computation for greedy action selection.

This saving and reloading mechanism is separate from the training-step update. During onboard training, the working parameters are updated inside the accelerator; file storage is used only after training or before a later run.

5 Evaluation and Results

This chapter evaluates the implemented onboard DQN training system. The evaluation focuses on the learning behaviour of the trained policy, the control performance after training, the runtime difference between the processor-only and FPGA-accelerated implementations, and the FPGA resource usage of the implemented network variants.

The main purpose of the runtime evaluation is to explain why the FPGA implementation reduces the complete training time compared with the CPU reference. Therefore, the results are interpreted together with the implementation structure described in Chapter 4, especially the accelerator-side execution of repeated DQN computations.

5.1 Evaluation Setup

The evaluation uses the embedded FPGA-based DQN training system described in Chapter 4. The inverted-pendulum task is used as the control benchmark for training and evaluation.

The evaluation has three goals. The first goal is to verify that the DQN agent can be trained onboard on the ZCU104 platform. The second goal is to check whether the trained policy can control the inverted pendulum after training. The third goal is to compare the complete onboard training runtime with a CPU-based software reference using the same training configuration.

For the onboard implementation, the processor-side and accelerator-side roles follow the implementation described in Chapter 4. The CPU reference uses the same training setup, but executes the complete training procedure in software.

Table 5.1 summarizes the evaluation setup and the DQN configuration used for the experiments.

Tab. 5.1: Evaluation setup and DQN configuration

Parameter	Value
Target platform	ZCU104 embedded FPGA platform
FPGA device family	Zynq UltraScale+ MPSoC
FPGA clock frequency	100 MHz
CPU reference platform	12th Gen Intel Core i5-1235U
State representation	$(\cos \theta, \sin \theta, \dot{\theta})$
Action set	$[-2.0, -0.5, -0.1, 0.0, 0.1, 0.5, 2.0]$
Network architecture	$3 \rightarrow 32 \rightarrow 32 \rightarrow 7$
Hidden layers	2
Neurons per hidden layer	32
Hidden-layer activation	ReLU
Output layer	Linear Q-value output
Replay-buffer capacity	20000 transitions
Replay-buffer sample size	64
Learning-start threshold	1000 stored transitions
Training frequency	Every 4 environment steps
Target-network update interval	Every 1000 environment steps
Discount factor, γ	0.99
Learning rate	0.0001
Optimizer	Adam
Training episodes	8000
Episode length	250 steps
Total training steps	2,000,000
Training action selection	ϵ -greedy
Validation and evaluation action selection	Greedy

The same DQN configuration is used for the CPU reference and the onboard FPGA implementation. Therefore, the runtime comparison focuses on the execution platform rather than on differences in the learning workload. During validation and final evaluation, exploration is disabled and the trained main network is used without parameter updates.

5.2 Learning Behaviour and Control Evaluation

The learning behaviour of the onboard DQN training process is evaluated using the episode reward recorded during training. The trained policy is then evaluated with exploration disabled to check whether it can control the inverted pendulum from different initial

conditions. Control performance is reported using accumulated reward, stable steps, and stable-step ratio.

Action selection is done through an ϵ -greedy method during training. Therefore, the episode reward can fluctuate because exploratory actions are still taken while the policy is being updated. For this reason, the training reward is used mainly to observe the learning trend. The final control behaviour is evaluated separately using greedy action selection and fixed network parameters.

The episode reward is calculated as the sum of the rewards obtained during one episode:

$$r_{\text{episode}} = \sum_{t=1}^T r_t, \quad (5.1)$$

where T is the episode length. In this implementation, each episode contains 250 steps. A higher episode reward indicates that the pendulum remains closer to the desired operating region for a longer part of the episode.

Figure 5.1 shows one representative onboard training run. The light curve shows the episode reward, while the darker curve shows the 50-episode moving average. The moving average is used only to make the learning trend easier to observe.



Fig. 5.1: Representative single-seed training reward during onboard DQN training.

At the beginning of training, the reward is low because the network parameters have not yet been adapted to the task. During training, the reward increases as the accelerator updates the main network from sampled replay-buffer transitions. The remaining variation is expected because exploration remains active during training.

For the final control evaluation, a step is counted as stable when the pendulum satisfies

$$|e_t| < 0.35 \text{ rad} \quad \text{and} \quad |\dot{\theta}_t| < 0.5 \frac{\text{rad}}{\text{s}}. \quad (5.2)$$

The stable-step ratio is calculated as

$$\eta_{\text{stable}} = \frac{N_{\text{stable}}}{N_{\text{total}}}, \quad (5.3)$$

where N_{stable} is the number of stable steps and N_{total} is the total number of evaluated steps. This metric directly reports how much of the evaluation time the trained policy keeps the pendulum inside the defined stable region.

After training, the representative policy is evaluated using greedy action selection. Exploration is disabled, and the network parameters are kept fixed. The policy is tested across five initial conditions, as shown in Table 5.2. Each case is evaluated for 250 time steps, giving 1250 evaluated steps in total.

Tab. 5.2: Initial conditions used for final evaluation

Case	Initial angle θ_0	Initial angular velocity $\dot{\theta}_0$
1	π	0.0
2	$\pi + 0.20$	-2.0
3	1.67	2.0
4	1.67	-1.0
5	$\pi + 1.67$	0.0

Figure 5.2 shows the evaluation reward obtained for each initial condition.

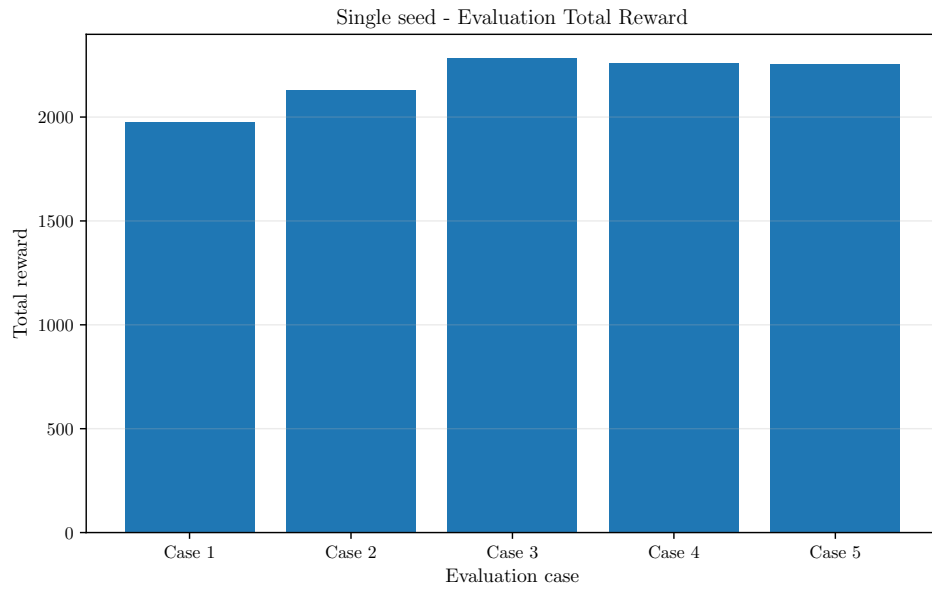


Fig. 5.2: Evaluation reward of one trained policy across five initial conditions.

Figure 5.3 shows the corresponding number of stable steps for the same evaluation cases.

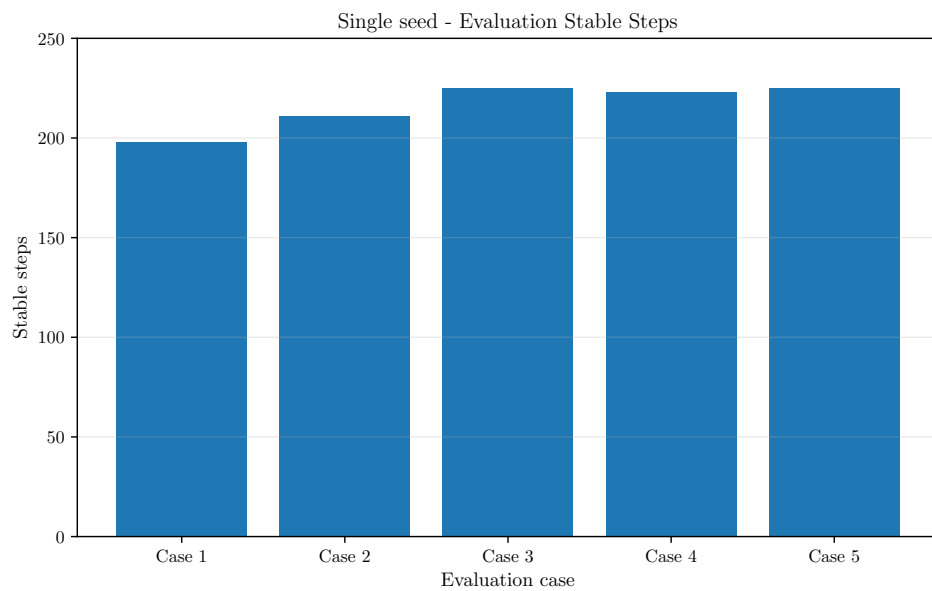


Fig. 5.3: Stable steps of one trained policy across five initial conditions.

Across the five evaluation cases, the trained policy reaches 1082 stable steps out of 1250 evaluated steps. This corresponds to a stable-step ratio of 0.866. The average reward across the same evaluation cases is 2179.938.

Tab. 5.3: Final control-performance evaluation of the representative trained policy

Metric	Result
Average reward	2179.938
Stable steps (Total 1250)	1082
Stable-step ratio	0.866

The representative evaluation shows that the trained policy achieves effective control across all tested initial conditions. The reward values and stable-step count confirm the final control performance of the representative onboard training run.

The onboard training experiment is also repeated across multiple random seeds. This checks whether the observed learning behaviour is consistent and not only caused by one initialization or exploration sequence. For each episode index, the mean training reward across the repeated runs is calculated as

$$\bar{r}_{\text{train}}(e) = \frac{1}{N} \sum_{i=1}^N r_{\text{train},i}(e), \quad (5.4)$$

where e is the episode index, N is the number of seeds, and $r_{\text{train},i}(e)$ is the training reward of seed i at episode e .

Figure 5.4 shows the multi-seed training reward during onboard DQN training. The mean reward increases during training and reaches a high-reward region after the initial learning phase. The wider standard-deviation region during the transition phase shows that different seeds learn at slightly different rates. After convergence, the variation becomes smaller, which indicates more consistent behaviour between the trained policies.



Fig. 5.4: Multi-seed training reward during onboard DQN training.

The solid line in Figure 5.4 shows the mean training reward across all seeds. The shaded region represents one standard deviation above and below this mean. Standard deviation measures how strongly the training rewards from different seeds vary around the mean value. It is calculated for each episode e as

$$\sigma_{\text{train}}(e) = \sqrt{\frac{1}{N} \sum_{i=1}^N (r_{\text{train},i}(e) - \bar{r}_{\text{train}}(e))^2}. \quad (5.5)$$

After training, each trained policy is evaluated with greedy action selection. For each seed, the evaluation reward is averaged over the five evaluation cases:

$$\bar{r}_{\text{eval},i} = \frac{1}{M} \sum_{j=1}^M r_{\text{eval},i,j}, \quad (5.6)$$

where M is the number of evaluation cases and $r_{\text{eval},i,j}$ is the evaluation reward of seed i in evaluation case j .

Figure 5.5 shows the average evaluation reward for each trained seed.

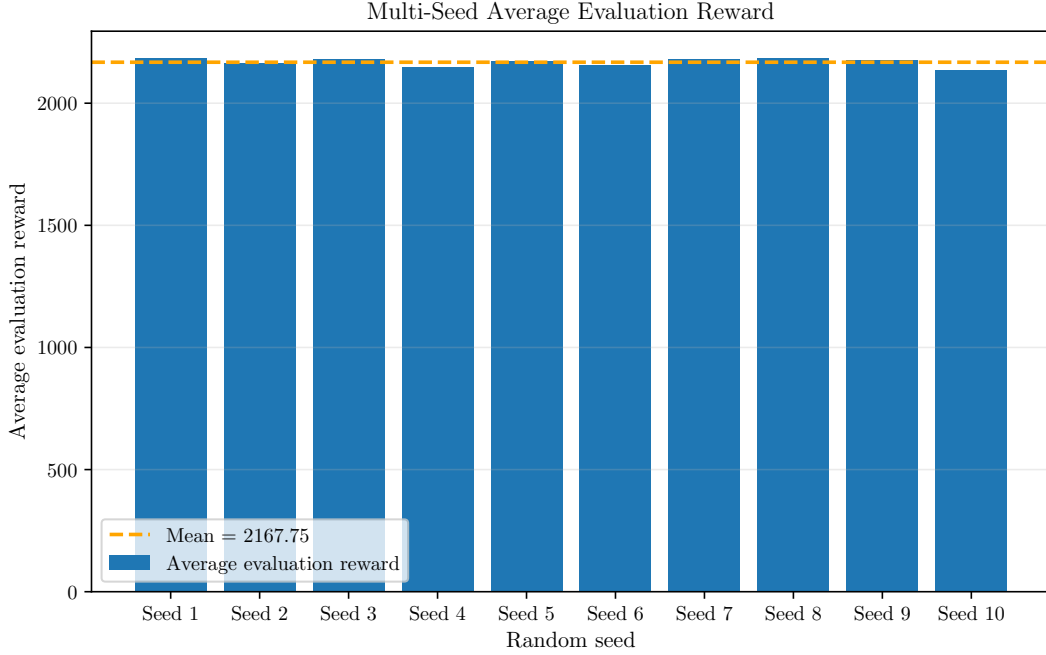


Fig. 5.5: Multi-seed average evaluation reward after onboard DQN training.

The evaluation rewards remain close to the mean value across the different seeds. The mean evaluation reward is 2167.75, which shows that the onboard training process produces similar final reward performance across repeated runs.

For each seed, the stable-step ratio is calculated over all evaluation cases as

$$\eta_{\text{stable},i} = \frac{N_{\text{stable},i}}{N_{\text{total},i}}, \quad (5.7)$$

where $N_{\text{stable},i}$ is the number of stable steps obtained by the policy trained with seed i , and $N_{\text{total},i}$ is the total number of evaluated steps for that seed.

The mean stable-step ratio across all seeds is calculated as

$$\bar{\eta}_{\text{stable}} = \frac{1}{N} \sum_{i=1}^N \eta_{\text{stable},i}, \quad (5.8)$$

where N is the number of repeated training seeds.

Figure 5.6 shows the stable-step ratio for each trained seed.

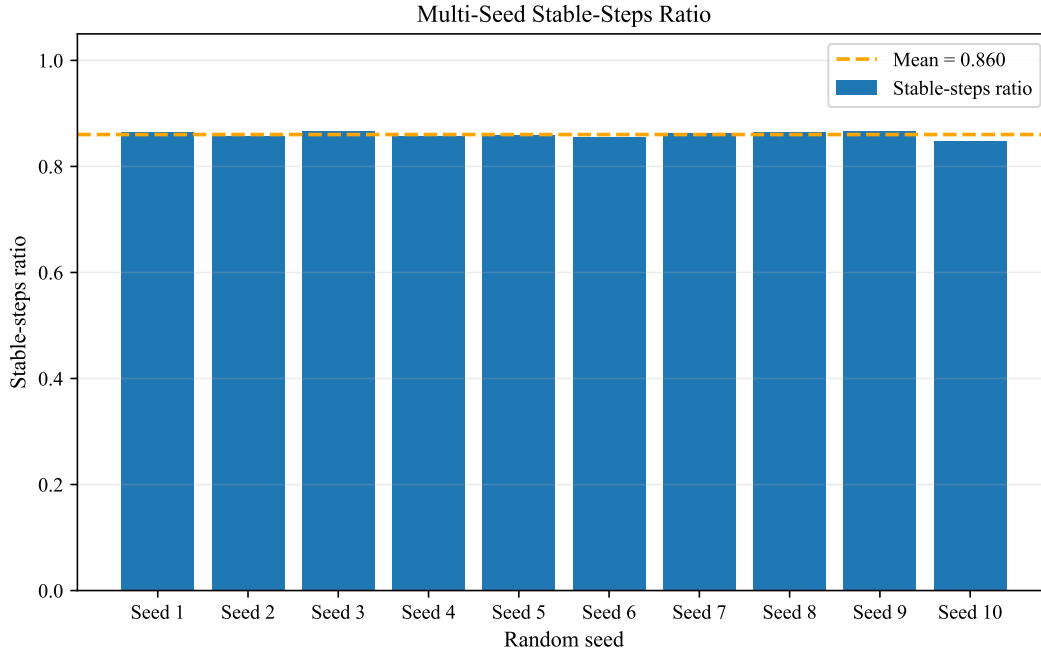


Fig. 5.6: Multi-seed stable-step ratio after onboard DQN training.

The stable-step ratios remain close across the repeated seeds. This shows that the trained policies achieve similar stability behaviour after onboard training. Since the stable-step ratio is based directly on the deterministic stability condition, it is suitable for evaluating how consistently the trained policies keep the pendulum inside the desired operating range.

5.3 Runtime Comparison and Speedup Analysis

The runtime comparison evaluates the complete training runtime of the processor-only reference and the onboard FPGA implementation. Both implementations use the same training configuration described in Section 5.1. The number of episodes, episode length, replay-buffer size, replay-sample group size, training interval, target-network update interval, action set, and network architecture are kept the same. Therefore, the comparison does not measure a simplified training task, but the effect of executing the same DQN workload on different platforms. The reported runtimes were measured for one complete training run of each implementation using this configuration.

The processor-only reference was executed on a 12th Gen Intel Core i5-1235U processor. The system provides 10 physical cores and 12 logical processors, and the reported base clock frequency is 1.30 GHz. The onboard implementation was executed on the ZCU104 platform, where the programmable-logic accelerator was operated at 100 MHz. These clock frequencies are not compared directly, because the CPU and FPGA execute the

workload with different execution models. The CPU executes the training procedure as software instructions, while the FPGA implementation maps repeated training operations to dedicated hardware datapaths.

The processor-only reference required 14 775.000 s for the complete training run, while the onboard FPGA implementation required 1093.765 s. The speedup is therefore

$$\text{Speedup} = \frac{14775.000}{1093.765} = 13.51. \quad (5.9)$$

Table 5.4 compares the measured training runtime with the final stable-step result. The final evaluation was performed using the same five evaluation cases, with 250 steps per case.

Tab. 5.4: Runtime and final control-performance comparison

Metric	CPU reference	Onboard FPGA
Complete training runtime	14 775.000 s	1093.765 s
Stable steps (Total 1250)	1081	1082
Stable-step ratio	0.865	0.866

The stable-step results are almost identical for both implementations. Therefore, the runtime reduction is not obtained by reducing the learning task or accepting a weaker final policy.

The runtime reduction is mainly caused by how the repeated DQN training operations are executed. In the processor-only reference, the forward pass, target-value calculation, TD-error calculation, backpropagation, gradient calculation, Adam update, and target-network synchronization are executed as software operations. In the onboard implementation, these operations are implemented as hardware routines inside the programmable-logic accelerator.

The accelerator keeps the main-network parameters, target-network parameters, replay buffer, intermediate activations, gradients, and Adam optimizer states inside programmable logic. During onboard training, the processor therefore does not transfer complete replay memory sample groups, network parameters, gradients, or optimizer states for each update. It sends only the step result and control values through the register interface, while the accelerator performs the repeated training update internally.

A major reason for the runtime improvement is the parallel scheduling of the main-network and target-network forward computations. The current state from the replay buffer is computed by the main network at each training iteration, whereas the next state is computed by the target network. Since both networks use separate parameter storage, these two forward paths can be scheduled as separate hardware tasks. This allows the current-state Q-values and next-state target Q-values to be produced with

overlap in hardware, instead of being executed strictly one after the other as software function calls.

Loop pipelining increases the throughput of the repeated matrix-vector operations used in the forward paths and parameter-update path. The hardware implementation can start a new operation before the previous operation has fully completed. Partial loop unrolling with a parallel factor of 2 allows selected neuron, action, and parameter operations to be processed in pairs where the memory access pattern allows it. This does not make the complete neural network fully parallel, but it increases the amount of parallel work compared with a purely sequential software loop.

Array partitioning supports this partial parallelism. The small state arrays are completely partitioned, so the three state values ($\cos \theta, \sin \theta, \dot{\theta}$) can be accessed independently. The hidden-layer buffers, Q-value arrays, error buffers, weight arrays, target-network arrays, and Adam moment arrays are cyclically partitioned with a factor of 2. This provides additional memory access paths and reduces memory-port conflicts during unrolled operations.

The backward computation reuses intermediate values saved during the main-network forward pass. The hidden-layer pre-activation and activation values are kept in local buffers and are reused for the ReLU-gradient and gradient calculations. The output error is applied only to the Q-value of the sampled action, and the hidden-layer errors are propagated through the locally stored weights. The Adam update is then performed locally using the trainable parameter and its corresponding first- and second-moment values.

The replay buffer and target-network synchronization are handled inside the accelerator to reduce data movement between the processor and programmable logic. During each training update, the accelerator reads the sampled transitions directly from its local replay buffer instead of receiving individual states, actions, rewards, and next states through repeated processor-controlled transfers. Mapping the large replay-state arrays to URAM provides sufficient on-chip storage capacity for this local access. This avoids communication overhead and allows replay sampling and training computation to proceed without external-memory accesses for each sampled transition. Similarly, the main-network and target-network parameters are stored locally in the accelerator. When the target-update interval is reached, the target parameters are copied directly from the main-network arrays using pipelined operations. Although this copy still requires accelerator clock cycles, it avoids transferring the complete parameter set from the accelerator to the processor and back again. Therefore, replay-buffer access and target-network synchronization add less overhead to the complete training loop, contributing to the reduced onboard training runtime.

Figure 5.7 shows the runtime breakdown used to interpret the measured speedup. The measured totals are 14 775.000 s for the CPU reference and 1093.765 s for the onboard FPGA implementation. The operations are grouped into four categories that correspond to the main stages of the implemented training flow.

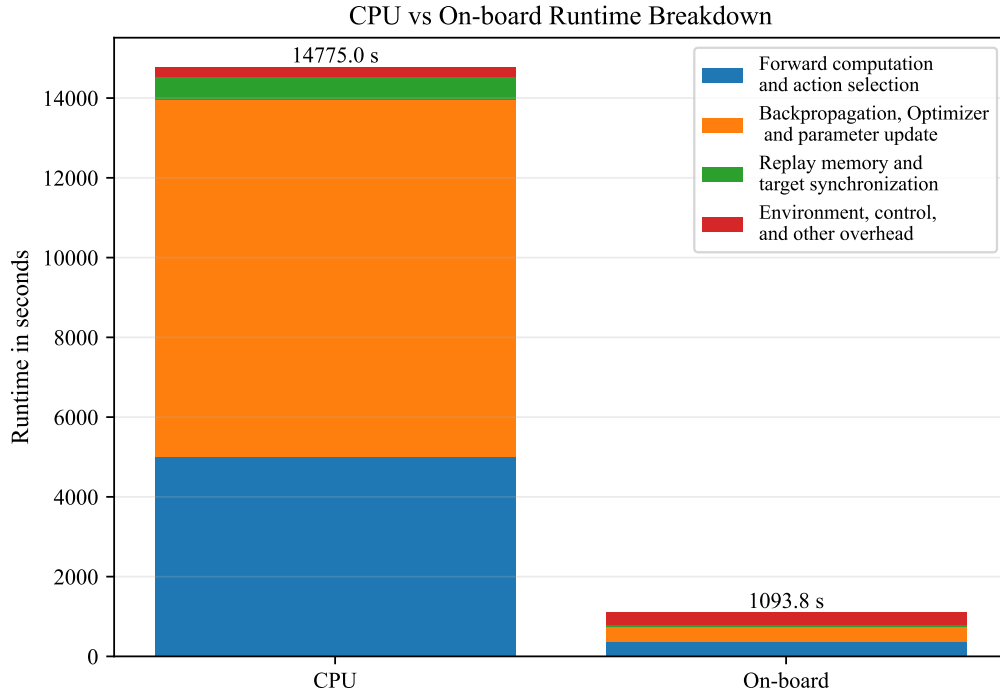


Fig. 5.7: Stage-level runtime breakdown of the CPU reference and onboard FPGA implementation.

The category *Forward computation and action selection* contains the main-network and target-network forward paths together with greedy and ϵ -greedy action selection. The category *Backpropagation and parameter update* contains hidden-layer error calculation, gradient calculation, and Adam-based weight and bias updates. The category *Replay buffer and target synchronization* contains transition storage, replay sampling, and copying main-network parameters to the target-network arrays. The category *Environment, control, and other overhead* contains processor-side work such as environment execution, reward calculation, episode control, AXI-Lite register access, validation, logging, and measurement overhead.

The runtime breakdown is derived from the measured execution times and grouped according to the main stages of the implemented training flow. The purpose of this grouping is to make the dominant runtime contributors visible and to relate them to the accelerator structure. This breakdown supports the measured speedup by showing that the main runtime reduction occurs in the repeated DQN computation stages, while environment execution and processor-side control remain part of the onboard runtime.

5.4 FPGA Resource Usage and Network Depth

The size of the neural network directly impacts the hardware cost of the DQN accelerator. Each additional layer increases the number of parameters, intermediate values, and arithmetic operations that must be implemented in programmable logic. Therefore, different network depths were tested to observe the relation between execution time, resource utilization, and control stability.

Table 5.5 shows the runtime, stable-step ratio, and resource utilization for the tested network configurations. Resource utilization is reported as a percentage of the total available ZCU104 resources.

Tab. 5.5: Runtime, stable-step ratio, and resource utilization for different network configurations

Network	Runtime	Stable-step Ratio	LUT	FF	BRAM	URAM	DSP
1 layer	451.313 s	0.641	7.15 %	3.45 %	11.80 %	50.72 %	11.87 %
2 layers	1093.765 s	0.866	10.62 %	5.17 %	14.22 %	50.72 %	21.60 %
3 layers	1716.738 s	0.864	14.35 %	6.96 %	16.64 %	50.72 %	31.73 %

The one-layer network has the shortest runtime and uses the fewest logic and DSP resources, but its lower stable-step ratio indicates weaker stable-control behaviour. This suggests that the smaller network does not provide enough representation capacity for the inverted-pendulum task. Increasing the depth to two layers improves the stable-step ratio, but also increases runtime and resource utilization.

The three-layer network has the highest runtime and resource usage. Although it provides more parameters and computation capacity, its stable-step ratio is almost the same as the two-layer network. In this comparison, the additional layer increases the hardware cost without giving a clear improvement in the final stability metric.

URAM utilization remains unchanged across the tested configurations because the replay buffer dominates the URAM usage. In contrast, LUT, FF, BRAM, and DSP utilization increase with network depth because additional layers require more arithmetic logic, parameter storage, and intermediate values.

The results show the trade-off between network depth, execution time, hardware usage, and stable-control performance. Smaller networks reduce runtime and resource usage but may provide weaker control stability. Deeper networks increase the training runtime and FPGA resource usage, but the three-layer result does not improve the stable-step ratio compared with the two-layer result.

6 Conclusion and Future Scope

6.1 Conclusion

This thesis presented an embedded FPGA-based implementation for onboard DQN training. The goal was to investigate whether the repeated computations of a reinforcement-learning training loop can be moved from a general-purpose processor to programmable logic and executed efficiently on an embedded platform. The inverted-pendulum task was used as the control benchmark.

The implemented system follows a hardware/software co-design approach. The processor-side software runs the environment simulation, reward calculation, episode handling, validation, evaluation, logging, and file handling. The programmable logic contains the custom DQN accelerator, including the main network, target network, replay buffer, target-value calculation, backpropagation, Adam optimizer update, target-network synchronization, action selection, and parameter transfer.

The results show that onboard DQN training was successfully executed on the ZCU104 platform. The trained policy was able to control the inverted pendulum after training. In the final evaluation, the policy achieved an average reward of 2179.938, 1082 stable steps out of 1250 evaluated steps, and a stable-step ratio of 0.866.

The runtime comparison showed that the onboard FPGA implementation reduced the complete training time from 14775.000s on the CPU reference to 1093.765s on the ZCU104 platform. This corresponds to a speedup of $13.51\times$. The reduction is mainly caused by the hardware-oriented execution of the repeated DQN update using pipelining, selective parallelism, array partitioning, local storage of parameters and optimizer states, and overlapping of suitable computations.

The resource and network comparison showed the trade-off between network depth, runtime, resource usage, and stable-control performance. Smaller networks reduced execution time and resource usage but provided weaker stable-control behaviour. Deeper networks increased runtime and resource usage without giving a clear improvement in the final stability metric for this task.

Overall, the work demonstrates that an embedded FPGA platform can be used not only for inference, but also for onboard reinforcement-learning training in a control application.

6.2 Future Scope

Future work can first extend the approach to other control structures and benchmark problems. The inverted pendulum is a suitable first task because it is unstable, nonlinear, and widely used for control evaluation. However, additional systems with different dynamics, state dimensions, and action spaces are needed to understand how well the accelerator concept transfers beyond this specific setup.

A second direction is to evaluate the approach on a real test bench under real-time constraints. In this thesis, the control task is evaluated using a simulated environment. A physical setup would introduce practical effects such as sensor noise, actuator limits, communication delay, timing jitter, and external disturbances. Testing on real hardware would therefore help to evaluate whether the trained or trainable policy remains stable and reliable under realistic operating conditions.

A third direction is to further optimize the FPGA implementation and study how far the design can be generalized. The current implementation uses specific choices for the network structure, memory organization, data handling, and hardware optimizations. Future work could investigate more reusable accelerator structures that can be adapted to different control tasks with less manual redesign. Automated design-space exploration could also be used to study trade-offs between runtime, resource usage, numerical precision, and control performance.

Appendix

A.1 Artificial Intelligence Use Declaration

To ensure transparency, the use of artificial intelligence tools during the preparation of this thesis is declared in Table A.1. The AI tools were used as supporting tools for drafting, language improvement and programming assistance. The final content, technical decisions, implementation, experiments, results, and interpretation were reviewed and verified by the author.

Tab. A.1: AI use declaration

Tool	Use Case	Scope
ChatGPT 5.5	Text drafting, grammar correction and language improvement	Thesis chapters and manuscript text
ChatGPT 5.5	C/C++ programming assistance for Vitis HLS implementation	DQN accelerator implementation and code refinement
Gemini 3.6	C/C++ programming assistance for Vitis HLS implementation	DQN accelerator implementation and code refinement
ChatGPT 5.5	Python programming assistance	PYNQ control scripts

Lists

List of Tables

2.1	Common HLS directives	11
2.2	Main FPGA hardware resources	15
3.1	Focused comparison of related work and research gap	19
4.1	Main accelerator operating modes	24
4.2	Main platform details used in the implementation	28
5.1	Evaluation setup and DQN configuration	35
5.2	Initial conditions used for final evaluation	37
5.3	Final control-performance evaluation of the representative trained policy	39
5.4	Runtime and final control-performance comparison	43
5.5	Runtime, stable-step ratio, and resource utilization for different network configurations	46
A.1	AI use declaration	49

List of Figures

2.1	Basic RL interaction between agent and environment.	4
2.2	Inverted pendulum model	5
2.3	Implemented Q-network mapping the three-component pendulum state to one Q-value for each of the seven discrete torque actions.	7
2.4	DQN training workflow with replay buffer, target network, and parameter update. Adapted from [9].	10
2.5	Sequential Execution	12
2.6	Task Parallelism within a Run	12
2.7	Task Parallelism with Pipelining	12
2.8	Conceptual illustration of sequential execution, task parallelism, and task parallelism with pipelining.	12
2.9	Loop Execution without Unrolling	13
2.10	Loop Unrolling with Unroll Factor 2	13

2.11	Conceptual illustration of rolled loop execution and loop unrolling with an unroll factor of two.	13
2.12	Conceptual illustration of block, cyclic, and complete array partitioning [11].	13
4.1	Overview of the hardware/software structure used for embedded DQN training.	22
4.2	ZCU104 evaluation board with implementation-relevant components marked: (1) Zynq UltraScale+ MPSoC, (2) PS-side DDR4 memory, (3) SD card connector, (4) USB/JTAG programming connection, (5) mode switch, (6) power switch, (7) power input, and (8) Ethernet/host connection.	27
4.3	Vivado block design connecting the Zynq processing system with the custom DQN accelerator.	29
5.1	Representative single-seed training reward during onboard DQN training. .	36
5.2	Evaluation reward of one trained policy across five initial conditions. . . .	38
5.3	Stable steps of one trained policy across five initial conditions.	38
5.4	Multi-seed training reward during onboard DQN training.	40
5.5	Multi-seed average evaluation reward after onboard DQN training.	41
5.6	Multi-seed stable-step ratio after onboard DQN training.	42
5.7	Stage-level runtime breakdown	45

Acronyms

AI Artificial Intelligence

ANN Artificial Neural Network

API Application Programming Interface

AXI Advanced eXtensible Interface

AXI-Lite Advanced eXtensible Interface Lite

BRAM Block Random Access Memory

CPU Central Processing Unit

DDR Double Data Rate

DQN Deep Q-Network

DSP Digital Signal Processing

FF Flip-Flop

FPGA Field-Programmable Gate Array

GPU Graphics Processing Unit

HLS High-Level Synthesis
IP Intellectual Property
LQR Linear Quadratic Regulator
LUT Lookup Table
ML Machine Learning
MLP Multilayer Perceptron
MMIO Memory-Mapped Input/Output
MPSoC Multiprocessor System-on-Chip
PID Proportional-Integral-Derivative
PL Programmable Logic
PS Processing System
PYNQ Python Productivity for Zynq
ReLU Rectified Linear Unit
RL Reinforcement Learning
RTL Register Transfer Level
SD Secure Digital
TD Temporal Difference
URAM UltraRAM

Symbols

a Selected action
 $\mathcal{A}$ Action space
 $\varepsilon_{\text{Adam}}$ Small constant used to avoid division by zero in Adam
 m First-moment estimate in the Adam optimizer
 $\widehat{m}_t$ Bias-corrected first-moment estimate in Adam
 g_t Adam update direction at update step t
 v Second-moment estimate in the Adam optimizer
 $\widehat{v}_t$ Bias-corrected second-moment estimate in Adam

α	Learning rate
e_t	Wrapped pendulum-angle error
β_1	First-moment decay factor in the Adam optimizer
β_2	Second-moment decay factor in the Adam optimizer
d	Episode termination flag
n	Environment-step counter
r_{episode}	Accumulated reward over one episode
T	Episode length or number of time steps in one episode
ϵ	Exploration probability in ϵ -greedy action selection
$r_{\text{eval},i}$	Evaluation reward of seed i
γ	Discount factor
g	Gravitational acceleration
B	Number of replay samples in one training event
$\mathbf{w}$	Main-network parameter vector
$\dot{\theta}_{\text{max}}$	Maximum permitted angular velocity
$\bar{r}_{\text{eval}}$	Mean evaluation reward across repeated runs
$\bar{r}_{\text{train}}$	Mean training reward across repeated runs
a'	Possible next action
s'	Next state of the environment
N	Number of repeated runs or random seeds
l	Pendulum length
m_p	Pendulum mass
π	Policy
$q_{\text{main}}(s, a)$	Q-value estimated by the main network
$q_{\text{target}}(s', a')$	Q-value estimated by the target network
$q(s, a)$	Q-value for state s and action a
G_t	Discounted return from time step t
r	Reward received from the environment
$\mathcal{R}$	Reward function

b_t	Upright-region reward bonus
r_t	Reward at time step t
i	Index of a replay sample, evaluation case, or repeated run
s	Current state of the environment
$\mathcal{S}$	State space
$\mathbf{w}^-$	Target-network parameter vector
k_w	Target-network update interval
y	Target value used for DQN training
L_i	Squared temporal-difference loss
δ	Temporal-difference error
θ	Angle of the pendulum
θ_0	Initial angle of the pendulum
$\dot{\theta}$	Angular velocity of the pendulum
$\dot{\theta}_0$	Initial angular velocity of the pendulum
Δt	Simulation time step
t	Time-step index
u_t	Torque applied at time step t
$r_{\text{train},i}$	Training reward of seed i
$\mathcal{T}$	State-transition mechanism

References

- [1] V. Mnih, K. Kavukcuoglu, D. Silver, et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [2] S. Israilov, L. Fu, J. Sánchez-Rodríguez, F. Fusco, G. Allibert, and C. Raufaste, “Reinforcement learning approach to control an inverted pendulum: A general framework for educational purposes,” *PLOS ONE*, 2023.
- [3] U. Yildiran, *Adaptive control of an inverted pendulum by a reinforcement learning-based lqr method*, 2023.
- [4] R. S. Jamisola Jr. and E. P. Dadios, “Experimental identification of manipulator dynamics through the minimization of its natural oscillations,” *Journal of Advanced Computational Intelligence and Intelligent Informatics*, vol. 14, no. 1, pp. 39–45, 2010. DOI: 10.20965/jaciii.2010.p0039

- [5] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, pp. 279–292, 1992.
- [6] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015. DOI: 10.1038/nature14539
- [7] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, *Playing atari with deep reinforcement learning*, 2013. arXiv: 1312.5602 [cs.LG].
- [8] D. P. Kingma and J. Ba, *Adam: A method for stochastic optimization*, 2015. arXiv: 1412.6980 [cs.LG].
- [9] Interconnected Automation Systems, University of Siegen, *Reinforcement Learning Course*, https://github.com/IAS-Uni-Siegen/RL_course, Lecture notes and course material, accessed September 2026, 2026.
- [10] AMD Xilinx, *Zynq UltraScale+ Device Technical Reference Manual*, UG1085, Accessed September 2026, 2023.
- [11] AMD Xilinx, *Vitis High-Level Synthesis User Guide*, UG1399, Version 2022.2, accessed September 2026, 2022.
- [12] M. Schenke and S. Bukarov, *Technical report on reinforcement learning control on the lucas-nülle inverted pendulum*, 2024.
- [13] M. Safeea and P. Neto, “A q-learning approach to the continuous control problem of robot inverted pendulum balancing,” *Intelligent Systems with Applications*, 2024.
- [14] D. Bates, *A hybrid approach for reinforcement learning using virtual policy gradient for balancing an inverted pendulum*, 2021.
- [15] M. Schenke, W. Kirchgässner, and O. Wallscheid, “Controller design for electrical drives by deep reinforcement learning: A proof of concept,” *IEEE Transactions on Industrial Informatics*, 2020.
- [16] G. Book, A. Traue, P. Balakrishna, A. Brosch, M. Schenke, S. Hanke, W. Kirchgässner, and O. Wallscheid, “Transferring online reinforcement learning for electric motor control from simulation to real-world experiments,” *IEEE Open Journal of Power Electronics*, 2021.
- [17] M. Schenke, B. Haucke-Korber, and O. Wallscheid, “Finite-set direct torque control via edge-computing-assisted safe reinforcement learning for a permanent-magnet synchronous motor,” *IEEE Transactions on Power Electronics*, 2023.
- [18] K. Guo, S. Zeng, J. Yu, Y. Wang, and H. Yang, *A survey of fpga-based neural network inference accelerator*, 2017.
- [19] A. Shawahna, S. M. Sait, and A. El-Maleh, “Fpga-based accelerators of deep learning networks for learning and classification: A review,” *IEEE Access*, 2019.
- [20] A. G. Blaiech, K. B. Khalifa, C. Valderrama, M. A. C. Fernandes, and M. H. Bedoui, “A survey and taxonomy of fpga-based deep learning accelerators,” *Journal of Systems Architecture*, 2019.
- [21] T. Schindler and A. Dietz, *Real-time inference of neural networks on fpgas for motor control applications*, 2020.
- [22] M. Štajnbrikner, I. Valek, T. Matić, and M. Vranješ, “Fpga-based deep neural network implementation for handwritten digit recognition,” *Advances in Multimedia*, 2025.

- [23] J. Su, J. Liu, D. B. Thomas, and P. Y. K. Cheung, *Neural network based reinforcement learning acceleration on fpga platforms*, 2017.
- [24] N. Sutisna, A. M. R. Ilmy, I. Syafalni, R. Mulyawan, and T. Adiono, “Farane-q: Fast parallel and pipeline q-learning accelerator for configurable reinforcement learning soc,” *IEEE Access*, 2023.
- [25] H. Cho, P. Oh, J. Park, W. Jung, and J. Lee, “Fa3c: Fpga-accelerated deep reinforcement learning,” in *ASPLOS*, 2019.
- [26] F. Ge, G. Zhang, Z. Li, and F. Zhou, *A fpga accelerator of distributed a3c algorithm with optimal resource deployment*, 2024.
- [27] C.-W. Hu, J. Hu, and S. P. Khatri, *Td3lite: Fpga acceleration of reinforcement learning with structural and representation optimizations*, 2022.
- [28] T. Kadokawa et al., *Binarized p-network for deep reinforcement learning of robot control from raw images on fpga*, 2021.
- [29] Y. Liu et al., “Acceleration methods for deep reinforcement learning using parallel and distributed computing: A survey,” *ACM Computing Surveys*, 2023.
- [30] H. Watanabe, M. Tsukada, and H. Matsutani, *An fpga-based on-device reinforcement learning approach using online sequential learning*, 2023.
- [31] M. Rothmann and M. Porrmann, *Fpga-based acceleration of deep q-networks with stann-rl*, 2024.
- [32] Q. Deng, H. Yang, Y. Zhao, R. Li, and G. Dong, *Fpga-based heterogeneous computing framework for environment-level parallel decision-making in autonomous driving and robotic control*, 2026.